从单边雅可比 SVD 到 CUDA 数值系统

算法、并行化与工程边界

2025-2026春季学期并行程序设计 SIMD 实验报告

jacobi-svd-cuda

maintainer@example.com

2026 年 4 月 22 日

目录

1 SVD 是什么 . . . 3

1.1 几何图像：旋转、缩放、再旋转 . . . 3

1.2 为什么 SVD 到处都在 . . . 4

1.3 定理：任意实矩阵都存在 SVD . . . 5

2 单边雅可比算法实现 SVD . . . 6

2.1 SVD 算法谱系 . . . 7

2.2 为什么选择单边雅可比 . . . 8

2.3 数学模型 . . . 9

2.4 几何理解 . . . 10

2.5 伪代码 . . . 11

2.6 复杂度分析 . . . 12

3 在 GPU 上并行化与规约 . . . 14

3.1 数值正确性与验证接口 . . . 14

3.2 访存流量与 roofline 视角 . . . 15

3.3 将逻辑模型映射到体系结构实现 . . . 16

3.4 主机侧契约：RAII 管资源，kernel 管计算 . . . 17

3.5 内存布局、转置策略与自动调优 . . . 18

3.6 从列对到无冲突 round . . . 19

3.7 规约：局部 Gram 矩阵的并行化 . . . 19

3.8 kernel 管线与同步边界 . . . 20

3.9 执行配置与资源占用 . . . 22

3.10 当前实现的边界与后续方向 . . . 23

4 从 Kernel 到系统：纯函数构件与良构边界 . . . 23

4.1 边界的必要性：高性能代码最怕职责膨胀 . . . 24

4.2 轻量 DDD：领域不是业务，而是数值不变量 . . . 25

4.3 Kernel 的公开契约：窄入口，类型化配置，主机侧结果 . . . 26

4.4 外层支撑：让纯 kernel 落到真实系统 . . . 27

4.5 健壮性要求：所有权清楚，失败可翻译 . . . 28

4.6 实体生命周期：同名“矩阵”不是同一种东西 . . . 29

4.7 当前实现的边界风险 . . . 31

4.8 小结 . . . 31

5 将内存映射到文件进行流式处理 . . . 32

5.1 设计目标：把 I/O 从批量文件操作重构为矩阵流 . . . 32

5.2 数据模型与文件格式 . . . 33

5.3 内存映射：把文件访问交给虚拟内存系统 . . . 34

5.4 只读映射：不可变字节视图与游标推进 34
5.5 追加式输出映射：用容量管理替代频繁写系统调用 35
5.6 编码与解码：分块处理而不是整块临时复制 36
5.7 派发读取：把字节流先送入页锁定任务缓冲 36
5.8 Policy-based stream：把格式差异从控制流中剥离 38
5.9 错误模型：在边界处失败，而不是在深处腐化 39
5.10 性能模型：mmap 优化的是数据通路，而不是算法复杂度 39
5.11 与计算 pipeline 的接口边界 40
5.12 当前实现的边界与可演化方向 41
5.13 小结 42

6 全局线程池、重排序缓冲区与 Pipeline 聚合根 42
6.1 为什么这样设计 pipeline：理论动机、顺序契约与复杂度边界 43
6.2 全局线程池实现：把并发入口集中起来，而不是到处偷开线程 44
6.3 流程与重排序缓冲区：主线程派发、工作线程执行、写线程恢复顺序 45
6.4 工程健壮性细节：配置归一化、关闭协议、异常传播与不变量 47
6.5 当前实现的边界与可演化方向 49
6.6 小结 51

7 CLI 表示层：从用户意图到 Pipeline 契约 51
7.1 设计目标与表示层定位 51
7.2 命令空间与选项契约 52
7.3 执行链路：从 `argv` 到 `run_pipeline` 53
7.4 关键工程细节 54
7.5 当前实现的边界与可演化方向 55
7.6 本节小结 56

8 CMake 构建体系：把数值程序编译成可复现实验 56
8.1 构建系统的研究对象：从文件集合到可执行 artifact 57
8.2 工具链选择与构建契约 57
8.3 可复现实验的操作模型 61
8.4 工程审查与后续演化 63
8.5 小结 66

1 SVD 是什么

奇异值分解（Singular Value Decomposition, SVD）是线性代数里最像“显微镜”的分解：它不要求矩阵是方阵，不要求矩阵可逆，也不要求矩阵可对角化，却仍然能把任意线性映射

$$A : \mathbb{R}^n \to \mathbb{R}^m$$

拆成三个结构极其干净的部分：

$$A = U\Sigma V^T.$$

其中 $U \in \mathbb{R}^{m\times m}$ 与 $V \in \mathbb{R}^{n\times n}$ 是正交矩阵（orthogonal matrix），满足 $U^TU = I_m$、$V^TV = I_n$；$\Sigma \in \mathbb{R}^{m\times n}$ 是非负“对角”矩阵，即

$$\Sigma = \begin{bmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_p \\ & & & \end{bmatrix}, \qquad p = \min\{m, n\}, \qquad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0.$$

这些非负数 σ_i 称为 A 的奇异值（singular value）。U 的列向量称为左奇异向量（left singular vector），V 的列向量称为右奇异向量（right singular vector）。

如果矩阵的秩（rank）为 r，则恰有 r 个正奇异值。此时也常写成紧 SVD （thin SVD / economy SVD）：

$$A = U_r\Sigma_r V_r^T = \sum_{i=1}^{r} \sigma_i u_i v_i^T,$$

其中 $U_r = [u_1, \ldots, u_r] \in \mathbb{R}^{m\times r}$， $V_r = [v_1, \ldots, v_r] \in \mathbb{R}^{n\times r}$， $\Sigma_r = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$。这个求和形式非常关键：它说明矩阵可以被看作 r 个秩一矩阵（rank-one matrix）的叠加。每一项 $\sigma_i u_i v_i^T$ 都是一种方向性的能量模式；σ_i 越大，该模式越重要。

1.1 几何图像：旋转、缩放、再旋转

从几何上看，SVD 把任何线性映射拆成三步：

$$x \xrightarrow{V^T} V^Tx \xrightarrow{\Sigma} \Sigma V^Tx \xrightarrow{U} U\Sigma V^Tx.$$

第一步 V^T 只是改变输入空间中的正交坐标系；第二步 Σ 沿坐标轴缩放，并可能把某些方向压扁到 0；第三步 U 再改变输出空间中的正交坐标系。因此，SVD 的核心断言可以说成：

> 任意矩阵都可以被理解为：输入空间的一次正交重标定、坐标轴方向的非负缩放、输出空间的一次正交重标定。

这比“矩阵可以分解成三个矩阵”更有信息量。因为正交变换保持长度与角度，不引入数值放大；真正改变几何尺度的全部信息，都被集中到非负奇异值 σ_i 里。于是立即得到

$$\|A\|_2 = \sigma_1, \qquad \|A\|_F^2 = \sum_{i=1}^{p} \sigma_i^2, \qquad \kappa_2(A) = \frac{\sigma_1}{\sigma_r} \quad (A \text{ 满列秩或可逆时}),$$

其中 $\|\cdot\|_2$ 是谱范数（spectral norm），$\|\cdot\|_F$ 是 Frobenius 范数（Frobenius norm），κ_2 是二范数条件数（condition number）。这也是 SVD 在数值计算中拥有特殊地位的原因：它把“一个线性系统有多病态”这件事转化为奇异值谱（singular spectrum）的形状。

1.2 为什么 SVD 到处都在

SVD 的应用并不是偶然拼贴出来的；它们都来自同一个结构事实： SVD 同时给出矩阵的正交基、尺度谱与最优低秩逼近。下面这些场景看似分散，其实共享同一套数学骨架。

最小二乘与伪逆。 在线性最小二乘（linear least squares）问题

$$\min_x \|Ax - b\|_2$$

中，若 $A = U\Sigma V^T$，则 Moore–Penrose 伪逆（pseudoinverse）为

$$A^+ = V\Sigma^+ U^T, \qquad \Sigma^+_{ii} = \begin{cases} 1/\sigma_i, & \sigma_i > 0, \\ 0, & \sigma_i = 0. \end{cases}$$

于是最小范数解（minimum-norm solution）可以写成 $x_\star = A^+ b$。这一表达式把病态性完全暴露出来：当某些 σ_i 很小时，$1/\sigma_i$ 会放大噪声。因此截断 SVD（truncated SVD）和 Tikhonov 正则化（Tikhonov regularization）都可以被理解为对小奇异值方向的抑制。

低秩近似与压缩。 Eckart–Young–Mirsky 定理（Eckart–Young–Mirsky theorem）说明，若

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T,$$

则 A_k 是所有秩不超过 k 的矩阵里对 A 的最优逼近：

$$A_k \in \arg\min_{\operatorname{rank}(B)\le k} \|A - B\|_2, \qquad \|A - A_k\|_2 = \sigma_{k+1},$$

并且在 Frobenius 范数下也最优：

$$\|A - A_k\|_F = \left(\sum_{i=k+1}^{p} \sigma_i^2\right)^{1/2}.$$

图像压缩、推荐系统、潜在语义分析（latent semantic analysis）和矩阵补全（matrix completion）都在不同外衣下使用这一事实：保留大奇异值，丢弃小奇异值，就能保留主结构而过滤细节或噪声。

主成分分析。 主成分分析（Principal Component Analysis, PCA）本质上是中心化数据矩阵的 SVD。若 $X \in \mathbb{R}^{N\times d}$ 的每一行是一个样本，且各列已经去均值，则

$$X = U\Sigma V^T$$

中的 V 给出特征空间中的主方向，$\sigma_i^2/(N-1)$ 是样本协方差矩阵 $X^T X/(N-1)$ 的特征值。换句话说，PCA 不是另一门魔法；它是 SVD 对数据云几何形状的读谱。

信号、控制与科学计算。 在信号处理中，小奇异值往往对应低能量或噪声方向；在控制理论中，奇异值描述系统在不同方向上的增益；在偏微分方程（partial differential equation）离散化与反问题（inverse problem）中，奇异值衰减速度揭示问题的适定性（well-posedness）。在高性能计算里，SVD 又是许多鲁棒算法的基础构件：秩判定、子空间迭代、正交化、压缩存储和误差估计都绕不开它。

机器学习与大模型。 现代机器学习中，SVD 出现在权重矩阵压缩、低秩适配（Low-Rank Adaptation, LoRA）、谱归一化（spectral normalization）、表示分析、嵌入空间诊断等任务中。尤其当矩阵规模巨大时，完整 SVD 往往太贵，研究者会转向随机 SVD（randomized SVD）、块 Krylov 方法（block Krylov method）或 GPU 上的 Jacobi / QR / divide-and-conquer 变体。我们这个项目实现单边雅可比 SVD （one-sided Jacobi SVD），正是站在“高精度正交化”和“并行列变换”之间寻找工程平衡。

1.3 定理：任意实矩阵都存在 SVD

定理陈述。 对任意 $A \in \mathbb{R}^{m\times n}$，存在正交矩阵 $U \in \mathbb{R}^{m\times m}$、$V \in \mathbb{R}^{n\times n}$，以及非负对角矩阵 $\Sigma \in \mathbb{R}^{m\times n}$，使得

$$A = U\Sigma V^T.$$

若允许复矩阵 $A \in \mathbb{C}^{m\times n}$，同样结论成立，只需把转置 T 替换为共轭转置（conjugate transpose）$*$，把正交矩阵替换为酉矩阵（unitary matrix）。

证明思路。 真正需要证明的是：能否在输入空间中找到一组正交方向 v_i，使得 Av_i 在输出空间中也两两正交。若能做到，那么 Av_i 的长度就是奇异值，归一化后的 Av_i 就是左奇异向量。这个想法把问题从一般矩阵 A 拉回到对称半正定矩阵 A^TA，而后者可以被谱定理（spectral theorem）处理。

证明。 令

$$B = A^TA \in \mathbb{R}^{n\times n}.$$

则 B 是实对称矩阵，并且对任意 $x \in \mathbb{R}^n$，

$$x^TBx = x^TA^TAx = \|Ax\|_2^2 \geq 0,$$

所以 B 是半正定矩阵（positive semidefinite matrix）。由实对称矩阵谱定理，存在正交矩阵 $V = [v_1, \ldots, v_n]$ 和实数 $\lambda_1, \ldots, \lambda_n$，使得

$$Bv_i = \lambda_i v_i, \qquad V^TBV = \mathrm{diag}(\lambda_1, \ldots, \lambda_n).$$

又因为 B 半正定，所以

$$\lambda_i = v_i^TBv_i = \|Av_i\|_2^2 \geq 0.$$

将特征值按非增序排列：

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0, \qquad \sigma_i = \sqrt{\lambda_i}.$$

设 r 是正特征值个数，即

$$\sigma_1 \geq \cdots \geq \sigma_r > 0, \qquad \sigma_{r+1} = \cdots = \sigma_n = 0.$$

对 $1 \leq i \leq r$，定义

$$u_i = \frac{Av_i}{\sigma_i} \in \mathbb{R}^m.$$

我们先证明这些 u_i 两两正交且单位长度。对 $1 \leq i, j \leq r$，

$$u_i^T u_j = \frac{(Av_i)^T(Av_j)}{\sigma_i\sigma_j} = \frac{v_i^T A^T A v_j}{\sigma_i\sigma_j} = \frac{v_i^T B v_j}{\sigma_i\sigma_j} = \frac{\lambda_j v_i^T v_j}{\sigma_i\sigma_j} = \delta_{ij},$$

其中 δ_{ij} 是 Kronecker delta（Kronecker delta）。因此 $u_1, \ldots, u_r$ 是 $\mathbb{R}^m$ 中的一组标准正交向量。

接下来把它扩充为 $\mathbb{R}^m$ 的一组完整标准正交基：

$$u_1, \ldots, u_r, u_{r+1}, \ldots, u_m.$$

这种扩充可由 Gram–Schmidt 正交化（Gram–Schmidt orthogonalization）完成。令

$$U = [u_1, \ldots, u_m] \in \mathbb{R}^{m\times m}.$$

于是 U 是正交矩阵。

现在检查 AV 的列。对 $1 \leq i \leq r$，

$$Av_i = \sigma_i u_i.$$

对 $i > r$，由于 $\sigma_i = 0$，有

$$\|Av_i\|_2^2 = v_i^T A^T A v_i = v_i^T B v_i = \lambda_i = 0,$$

所以 $Av_i = 0$。因此

$$AV = [Av_1, \ldots, Av_n] = [\sigma_1 u_1, \ldots, \sigma_r u_r, 0, \ldots, 0] = U\Sigma,$$

其中 $\Sigma \in \mathbb{R}^{m\times n}$ 的前 $p = \min\{m, n\}$ 个对角元为 $\sigma_1, \ldots, \sigma_p$，其余元素为 0。右乘 V^T 得

$$A = U\Sigma V^T.$$

定理得证。

2 单边雅可比算法实现 SVD

上一章给出了奇异值分解（Singular Value Decomposition, SVD）的存在性结论：任意矩阵 $A \in \mathbb{R}^{m\times n}$ 都可以写成 $A = U\Sigma V^T$。然而，存在性本身并不构成可执行算法。数值线性代数更关心的是：在浮点数运算（floating-point arithmetic）中，如何以可控的舍入误差、可接受的计算复杂度，以及与目标硬件相匹配的访存模式，稳定地构造这一分解。

本章首先从算法谱系的角度定位主流 SVD 方法，随后说明本项目选择单边雅可比（one-sided Jacobi）路线的原因。之后，我们给出该方法的数学模型、几何解释与串行伪代码；后续章节将在此基础上讨论其 CUDA 并行化。

2.1 SVD 算法谱系

SVD 算法大体可以按“先规约再求解”与“直接正交化”两类思路理解。不同方法之间并不存在单一维度上的绝对优劣；它们分别在精度、吞吐、并行性、实现复杂度以及目标矩阵规模之间采取不同折中。

双对角化加 QR 迭代。 经典密集矩阵 SVD 通常先用 Householder 反射（Householder reflection）把 A 规约为双对角矩阵（bidiagonal matrix）：

$$A = QBP^T,$$

其中 Q、P 为正交矩阵，B 仅在主对角线与一条副对角线上非零。随后，对 B 执行隐式 QR 迭代（implicit QR iteration）或相关变体。该路线理论成熟、数值稳定，是 LAPACK 等基础线性代数库的重要实现路径。其主要局限在于：规约阶段与迭代阶段包含较强的数据依赖，直接映射到 GPU 时难以充分暴露细粒度并行性。

分治法。 分治 SVD（divide-and-conquer SVD）同样从双对角矩阵出发，将问题拆分为较小子问题，再通过秩一修正（rank-one modification）合并结果。对于需要全部奇异向量的中大型密集矩阵，该方法通常具有较高效率。相应代价是实现复杂度与工作空间需求较高，且合并阶段包含较精细的数值处理。

MRRR 与相关特征值路线。 部分实现会把 SVD 转化为对称特征值问题（symmetric eigenvalue problem），例如处理 A^TA 或增广矩阵

$$\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}.$$

MRRR（Multiple Relatively Robust Representations）等方法在特征值问题中具有重要地位。不过，显式形成 A^TA 会导致条件数平方化：

$$\kappa_2(A^TA) = \kappa_2(A)^2,$$

从而使小奇异值对应的方向更容易被舍入误差淹没。对于强调奇异向量鲁棒性的实现，这通常不是理想选择。

Krylov 与随机化方法。 当矩阵规模极大且只需要前 k 个奇异值或低秩近似时，Lanczos 双对角化（Lanczos bidiagonalization）、块 Krylov 方法（block Krylov method）与随机 SVD（randomized SVD）具有较强吸引力。它们并不追求完整分解，而是通过矩阵-向量乘法或矩阵-块乘法提取主子空间。这类方法适用于稀疏矩阵、大规模机器学习与近似压缩；但本项目的目标是实现一个结构清晰、结果完整且便于验证的 dense SVD kernel，因此它们不是本文的算法主线。

雅可比方法。 雅可比方法（Jacobi method）通过一系列二维正交旋转逐步消除非对角相关项。面向 SVD 时，常见形式包括双边雅可比（two-sided Jacobi）与单边雅可比：前者对对称矩阵或增广结构做左右旋转，后者直接旋转 A 的列，使列向量逐步正交。雅可比方法通常不是浮点操作次数最少的选择，但它具有两个对本项目尤其关键的性质：良好的正交性保持能力，以及高度局部的计算结构。对于 GPU，这种局部性意味着互不共享列的列对可以被组织为并行任务。

2.2 为什么选择单边雅可比

本项目选择单边雅可比，并不是因为它在所有问题规模上都具有最高吞吐，而是因为它与本项目的设计目标高度一致：避免不必要的条件数放大，直接控制列正交性，并提供可验证、可并行化的局部计算原语。

第一，避免显式形成 A^TA。 上一章的存在性证明借助 A^TA，但算法实现并不需要显式构造它。单边雅可比在每个列对 (p,q) 上只计算三个标量：

$$\alpha = a_p^T a_p, \qquad \beta = a_q^T a_q, \qquad \gamma = a_p^T a_q.$$

这等价于读取 2×2 的局部 Gram 矩阵（Gram matrix）信息，而不是构造完整的 A^TA。因此，该方法避免了条件数平方化所带来的系统性数值风险。

第二，正交性是算法的主目标，而不是副产品。 SVD 的核心不仅是获得奇异值，还包括构造高质量的左右奇异子空间。单边雅可比的每一步都直接降低列向量之间的相关性；收敛时，当前矩阵的列已近似两两正交。于是左奇异向量可由列归一化得到：

$$\sigma_j = \|a_j\|_2, \qquad u_j = a_j/\sigma_j.$$

该路线的状态含义十分明确：右奇异矩阵 V 由旋转累积而来，奇异值来自列范数，左奇异矩阵来自归一化后的工作矩阵列。

第三，旋转是局部原语。 一次 Jacobi 旋转只修改两列 a_p, a_q，以及右奇异矩阵中的两列 v_p, v_q。如果一轮中选出的列对互不重叠，例如

$$(1,2),(3,4),(5,6),\ldots,$$

则这些旋转之间不存在写冲突，可以并行执行。这个性质自然导向后续章节讨论的巡回赛排序（tournament ordering / round-robin ordering）：它把所有列对分批组织成无冲突的 matching。

第四，代码路径容易验证。 从工程角度看，单边雅可比的优势还在于状态空间有限且可观测。算法只需要维护当前工作矩阵 $A^{(k)}$、累积右旋转 $V^{(k)}$、容差 ε 与扫描轮次（sweep count）。每次旋转之后，可以检查局部 Gram 矩阵的非对角项是否下降；每个 sweep 之后，可以检查整体列正交残差。这使得教学推导、单元测试以及 CUDA kernel 调试都具有清晰的验证接口。

2.3 数学模型

设初始矩阵为

$$A^{(0)} = A, \qquad V^{(0)} = I_n.$$

单边雅可比维护如下不变量：

$$A^{(k)} = AV^{(k)},$$

其中 $V^{(k)}$ 是若干个平面旋转（plane rotation）的乘积，因此始终保持正交。当 $A^{(k)}$ 的列两两正交时，有

$$\left(A^{(k)}\right)^T A^{(k)} = \left(V^{(k)}\right)^T A^T A V^{(k)} = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_n^2),$$

于是 $V^{(k)}$ 的列给出右奇异向量，$A^{(k)}$ 的列范数给出奇异值。

现在考虑一对列 (p, q)。记

$$a = a_p, \qquad b = a_q,$$

并定义局部 Gram 矩阵

$$H = \begin{bmatrix} \alpha & \gamma \\ \gamma & \beta \end{bmatrix} = \begin{bmatrix} a^T a & a^T b \\ a^T b & b^T b \end{bmatrix}.$$

目标是构造一个二维正交旋转

$$G = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}, \qquad c^2 + s^2 = 1,$$

使更新后的两列

$$[a_p', a_q'] = [a_p, a_q]G$$

满足

$$(a_p')^T a_q' = 0.$$

等价地，要求

$$G^T H G$$

的非对角元为 0。

令 $t = s/c$。直接展开可得非对角元为

$$\gamma(c^2 - s^2) + (\alpha - \beta)cs.$$

若 $\gamma \neq 0$，令该项为 0，得到

$$\gamma t^2 + (\beta - \alpha)t - \gamma = 0.$$

设

$$\tau = \frac{\beta - \alpha}{2\gamma},$$

则

$$t^2 + 2\tau t - 1 = 0.$$

为避免在 $|\tau|$ 很大时发生灾难性抵消（catastrophic cancellation），采用稳定形式

$$t = \begin{cases} \dfrac{1}{\tau + \sqrt{1+\tau^2}}, & \tau \geq 0, \\ -\dfrac{1}{-\tau + \sqrt{1+\tau^2}}, & \tau < 0, \end{cases}$$

然后计算

$$c = \frac{1}{\sqrt{1+t^2}}, \qquad s = tc.$$

若 γ 已经足够小，则跳过旋转。实际实现中常采用相对准则

$$|\gamma| \leq \varepsilon\sqrt{\alpha\beta},$$

其中 ε 为容差。相比绝对阈值，该准则会随列范数自动缩放，因此在不同矩阵尺度下更加稳健。

一次旋转的列更新为

$$a'_p = ca_p - sa_q, \qquad a'_q = sa_p + ca_q,$$

右奇异向量矩阵同步更新为

$$v'_p = cv_p - sv_q, \qquad v'_q = sv_p + cv_q.$$

这里的 V 并不是后处理阶段补写的记录表，而是算法不变量的一部分：每次 A 右乘 G，V 也右乘同一个 G，从而始终保持 $A^{(k)} = AV^{(k)}$。

2.4 几何理解

单边雅可比的几何图像是：在输入空间中持续旋转坐标轴，使这些坐标轴经过 A 映射后，在输出空间中逐步变为互相正交的向量。

对列对 (a_p, a_q) 而言，当前两个输入基方向 e_p, e_q 经过 $A^{(k)}$ 后成为输出向量 a_p, a_q。若 $a_p^T a_q \neq 0$，说明这两个输出方向尚未正交。Jacobi 旋转并不直接在输出空间中修改这两个向量的几何关系，而是在输入平面 $\mathrm{span}\{e_p, e_q\}$ 内选择一组新的正交坐标：

$$e'_p = ce_p - se_q, \qquad e'_q = se_p + ce_q.$$

在新坐标下，

$$Ae'_p = a'_p, \qquad Ae'_q = a'_q,$$

且这两个输出向量被构造为正交。

因此，单边雅可比并不是对奇异向量进行启发式猜测；它是在逐步寻找一组输入正交基，使 A 将该基映射为一组输出正交向量。当所有列对都满足正交条件后，

$$AV = [\sigma_1 u_1, \ldots, \sigma_n u_n].$$

此时每个列向量的方向给出 u_i，长度给出 σ_i。该过程与上一章存在性证明的逻辑一致：证明中一次性调用谱定理，而算法中则通过一系列局部旋转逐步构造同一结构。

2.5 伪代码

下面的伪代码描述串行版本。CUDA 版本不会让线程任意竞争列资源，而是把列对组织为无冲突批次；但其数学核心仍然是同一个二维正交旋转。

```
algorithm OneSidedJacobiSVD(A, epsilon, max_sweeps)
input:
A: real m by n matrix
epsilon: relative orthogonality tolerance
max_sweeps: upper bound on sweeps
output:
U: left singular vectors
Sigma: singular values
V: right singular vectors

begin
V = Identity(n)
converged = false
sweeps = 0

while not converged and sweeps < max_sweeps do
converged = true
sweeps = sweeps + 1

for p = 0 to n - 2 do
for q = p + 1 to n - 1 do
alpha = dot(A[:, p], A[:, p])
beta  = dot(A[:, q], A[:, q])
gamma = dot(A[:, p], A[:, q])

threshold = epsilon * sqrt(alpha * beta)
if abs(gamma) > threshold then
converged = false

tau = (beta - alpha) / (2 * gamma)
if tau >= 0 then
t = 1 / (tau + sqrt(1 + tau * tau))
else
t = -1 / (-tau + sqrt(1 + tau * tau))
end if

c = 1 / sqrt(1 + t * t)
s = t * c

for i = 0 to m - 1 do
old_p = A[i, p]
```

```
    old_q = A[i, q]
    A[i, p] = c * old_p - s * old_q
    A[i, q] = s * old_p + c * old_q
    end for

    for i = 0 to n - 1 do
    old_p = V[i, p]
    old_q = V[i, q]
    V[i, p] = c * old_p - s * old_q
    V[i, q] = s * old_p + c * old_q
    end for
    end if
    end for
    end for
    end while

    for j = 0 to n - 1 do
    Sigma[j] = norm2(A[:, j])
    if Sigma[j] > epsilon then
    U[:, j] = A[:, j] / Sigma[j]
    else
    U[:, j] = ZeroVector(m)
    end if
    end for

    return U, Sigma, V
    end
```

该版本刻意保留朴素的双重列对循环，以便直接呈现算法不变量。后续 GPU 章节会把 (p,q) 的遍历方式替换为巡回赛排序，使同一批次中的列对互不重叠，从而将“局部旋转”提升为“批量并行旋转”。算法本体保持不变；改变的是调度方式与硬件映射。

2.6 复杂度分析

复杂度分析需要区分两个层次：一是固定一次 sweep 内的代价，二是算法达到给定容差所需的 sweep 数。前者可以由循环结构直接得到；后者依赖矩阵谱分布、奇异值间隔、初始列相关性以及容差 ε，通常不能仅由 m,n 给出一个紧致的先验界。因此，工程实现中通常以 `max_sweeps` 作为外部上界，并用列正交残差作为实际停止准则。

对一个固定列对 (p,q)，算法首先计算

$$\alpha = a_p^T a_p, \qquad \beta = a_q^T a_q, \qquad \gamma = a_p^T a_q.$$

这三个量可以在一次行扫描中同时累加，因此统计阶段的代价为 $O(m)$。若该列对已经满足

$$|\gamma| \le \varepsilon\sqrt{\alpha\beta},$$

则本次列对处理到此结束；若需要旋转，则还要更新 A 的两列，代价为 $O(m)$，并同步更新 V 的两列，代价为 $O(n)$。因此，一个列对的代价可以写成

$$O(m) + \mathbf{1}_{\mathrm{rot}}\, O(m+n),$$

其中 $\mathbf{1}_{\mathrm{rot}}$ 表示该列对是否实际执行旋转。

一次 sweep 会访问所有列对。列对总数为

$$N_p = \frac{n(n-1)}{2} = O(n^2).$$

若第 s 次 sweep 中实际旋转的列对数量为 r_s，则该 sweep 的代价为

$$O(mn^2) + O\big(r_s(m+n)\big), \qquad 0 \le r_s \le N_p.$$

在最坏情况下，几乎所有列对都需要旋转，于是单次 sweep 的上界为

$$O(mn^2 + n^3).$$

这里的 $O(mn^2)$ 来自列点积统计与工作矩阵 A 的列更新，$O(n^3)$ 来自右奇异矩阵 V 的累积更新。若只需要奇异值而不需要显式右奇异向量，则可以省略维护 V，相应的 $O(n^3)$ 项也会消失。但本项目输出完整 thin SVD，因此必须保留这一项。

设算法在给定容差下执行了 T 次 sweep。忽略初始化常数后，主迭代阶段的总时间复杂度为

$$O\big(T(mn^2 + n^3)\big).$$

收敛后的后处理需要计算每一列的范数并归一化：

$$\sigma_j = \|a_j\|_2, \qquad u_j = a_j/\sigma_j,$$

总成本为 $O(mn)$。因此完整串行算法的总体复杂度可写为

$$O\big(T(mn^2 + n^3) + mn\big),$$

其中 $T \le$ `max_sweeps`，但其实际值由输入矩阵和容差共同决定。

空间复杂度方面，工作矩阵 A 需要 $O(mn)$ 存储，右奇异矩阵 V 需要 $O(n^2)$，输出左奇异矩阵 U 需要 $O(mn)$，奇异值数组 Σ 需要 $O(n)$。若原地覆盖工作矩阵并把输出计入总空间，则存储复杂度为

$$O(mn + n^2).$$

对于 thin SVD 的典型场景 $m \ge n$，该空间界与输出规模同阶；也就是说，算法并没有引入超过结果本身数量级的额外长期存储。

这个复杂度结论也解释了为什么下一章要强调调度与规约。单边雅可比的数学原语很局部，但每个 sweep 覆盖 $O(n^2)$ 个列对；如果仍按串行顺序执行，局部性并不会自动转化为吞吐。GPU 并行化的核心价值，正是把同一 sweep 中互不冲突的列对组织为批量任务，在不破坏算法不变量的前提下压缩墙钟时间（wall-clock time）。

3 在 GPU 上并行化与规约

前两章已经建立了 SVD 的数学结构与单边雅可比（one-sided Jacobi）算法。本章转向 `kernels.cu` 的 CUDA 实现。不过这里的重点不是把串行伪代码机械搬进 kernel，而是解释一条更关键的翻译链：

$$\text{数值不变量} \longrightarrow \text{代价模型} \longrightarrow \text{体系结构映射} \longrightarrow \text{CUDA kernel 管线}.$$

单边雅可比以“列对”为基本数值操作；CUDA 则以 thread、block、warp、global memory、 shared memory 与 kernel launch 为基本执行结构。有效实现必须把这两套结构对齐：数学层要保护正交性、残差和收敛语义；机器层要控制访存流量、同步边界和资源占用。如果跳过这一步翻译，GPU 代码很容易变成“能跑但解释不了为什么这样跑”的实现。

3.1 数值正确性与验证接口

GPU 实现的第一问题不是性能，而是：并行化之后，算法语义是否仍然成立。对 SVD 而言，正确性不能只看程序是否运行结束，也不能要求 GPU 与 CPU 在每个元素上逐 bit 相等。更稳健的验证方式是回到数值不变量。给定输入矩阵 A 与输出 (U, Σ, V)，至少应检查：

$$\rho_A = \frac{\|A - U\Sigma V^T\|_F}{\max(1, \|A\|_F)},$$

$$\rho_U = \|U^T U - I\|_F, \qquad \rho_V = \|V^T V - I\|_F.$$

其中 ρ_A 是重构残差（reconstruction residual）。ρ_U, ρ_V 是左右奇异向量的正交性（orthogonality）误差。对于 thin SVD，$U \in \mathbb{R}^{m\times n}$，因此检查的是 $U^T U \approx I_n$，而不是 $UU^T \approx I_m$。

这些指标比逐元素比较更贴近算法本质。第一，奇异向量存在符号自由度：

$$u_j \leftarrow -u_j, \qquad v_j \leftarrow -v_j$$

不会改变 $U\Sigma V^T$。第二，GPU 规约（reduction）改变了浮点加法顺序。由于浮点加法不满足结合律（associativity），同一列点积在串行 CPU 循环和 CUDA 树形规约下出现末尾位差异是正常现象。因此，测试应使用容差（tolerance）和残差，而不是 bit-level equality。

合理的测试报告应记录：

$$(m, n), \quad \epsilon, \quad \texttt{max_sweeps}, \quad \texttt{sweeps}, \quad \rho_A, \quad \rho_U, \quad \rho_V.$$

若未来加入 CPU reference implementation，它适合作为 sanity check；但 GPU 与 CPU 的 U, V 不应被要求逐元素完全一致。更可靠的比较对象是奇异值、重构残差和子空间质量。

这里还要区分两种失败。达到 `max_sweeps` 仍未满足阈值，是数值算法状态； `cudaMalloc`、kernel launch 或 `cudaMemcpy` 失败，是 CUDA runtime 状态。前者应进入执行报告或验证报告，后者通常应终止当前计算并携带底层诊断信息。把二者混为“kernel 失败”会遮蔽真正原因。

3.2 访存流量与 roofline 视角

有了正确性目标之后，下一步才是性能判断。对 GPU kernel 而言，渐近工作量只能说明“做了多少数学操作”，不能说明“机器在哪里等待”。本实现最核心的代价来自全局内存（global memory）访问；布局策略决定这些访问是跨步地读列，还是先支付转置成本再连续地读列。

对一个列对 (p, q)，`pair_stats_kernel` 需要读取两列：

$$a_p, \quad a_q.$$

忽略缓存与统计量写回，读取量约为

$$2m \cdot \mathrm{sizeof}(\texttt{double}).$$

`apply_rotation_kernel` 需要读取并写回 A 的两列，再读取并写回 V 的两列，主导流量约为

$$4m \cdot \mathrm{sizeof}(\texttt{double}) + 4n \cdot \mathrm{sizeof}(\texttt{double}).$$

于是每个列对的主导 global memory traffic 可近似写成：

$$(6m + 4n) \cdot \mathrm{sizeof}(\texttt{double}).$$

一个 sweep 覆盖 $O(n^2)$ 个列对，因此流量规模近似为

$$O(mn^2 + n^3) \quad \text{个 double 级别的访存单元.}$$

这个数量级与算术工作量相同，但常数结构不同：Jacobi 旋转公式本身很轻，读写矩阵列却很重。

用 roofline model（屋顶线模型）来看，本 kernel 的算术强度（arithmetic intensity）并不高。统计阶段对每个元素只做少量乘加，应用阶段每个元素做一次二维旋转，却都要读写完整列。如果这些访问不能形成良好的合并访存（memory coalescing），实际性能上限会更接近显存带宽上限，而不是 FP64 峰值吞吐。

这解释了为什么布局不是细枝末节，而是设计依据。外部输入、输出和文件格式采用行主序（row-major）一维数组：

$$\mathrm{index}(i, j) = i \cdot n + j.$$

但单边雅可比天然按列工作。每次处理列对 (p, q) 时，线程需要读取

$$A[0, p], A[1, p], \ldots, A[m-1, p]$$

和

$$A[0, q], A[1, q], \ldots, A[m-1, q].$$

在行主序中，相邻行同一列的地址相隔 n 个 `double`，warp 中连续线程访问同一列时会形成跨步访问（strided access）。这正是算法逻辑与内存系统之间的张力：主机侧最自然的布局，并不必然是列规约最有利的布局。

若走直接路径，A 的列访问在行主序中跨步进行。若走布局转置路径，每个 testcase 额外增加两次 $O(mn)$ 的转置流量，却使 sweep 主体中的 A 列访问更接近连续访问。因此，布局转置收益大致来自：

saved bandwidth cost over sweeps > two transpose launches and extra $O(mn)$ traffic.

矩阵越大、sweep 越多、列访问越占主导，这个不等式越容易成立；矩阵越小、收敛越快，直接路径越可能更好。

3.3 将逻辑模型映射到体系结构实现

基于上述正确性与代价模型，单边雅可比到 CUDA 的映射可以概括如下：

数学对象或动作	CUDA 实现对象
矩阵 A, V, U, Σ	`DeviceMatrix` 管理的连续设备内存
外部矩阵格式与内部工作布局	行主序契约 + 可选列主序 `d_a_layout`
列对 (p, q)	一个 `int2`，一个 block 负责处理
局部 Gram 矩阵	α, β, γ 三个设备数组
列点积与列范数	block 内共享内存树形规约
Jacobi 旋转参数 (c, s)	每个列对一组设备数组元素
同一 round 的独立列对	多个 block 并行执行
round 间依赖	主机循环与 kernel launch 边界
sweep 收敛	设备 flag 与主机端判断
布局阈值选择	`LayoutTransposeMode` 与自动调优报告

这张表体现出高性能计算（High Performance Computing, HPC）中的核心原则：算法逻辑模型描述不变量、依赖关系与数值语义；体系结构实现处理数据运动、局部性（locality）、同步粒度（synchronization granularity）与资源占用。前者回答“什么计算在数学上等价”，后者回答“这种等价计算在机器上以什么代价发生”。

因此，CUDA 实现不是对伪代码的被动转录，而是一种受约束的重表达。数学模型中的“一次列对旋转”，在机器上会展开为读取两列、规约局部 Gram 矩阵、计算旋转参数、写回 A、写回 V、同步 round 边界、更新收敛状态等机制。反过来，体系结构也会塑造算法表达：如果某种遍历顺序导致严重的非合并访存或过密的全局同步，即使它在伪代码中最自然，也未必是合理的实现形式。

这里的张力可以分成三个互不重叠但共同完备的层面：

- **计算与数据运动。** SVD 公式看起来由浮点运算主导，但 GPU 上的上限常常先受显存带宽限制。
- **并行性与依赖。** 列对旋转具有局部性，但不同旋转仍可能共享列资源；调度必须先消除写冲突。

- **数值语义与执行顺序。** 树形规约改变浮点求和顺序，因此数学等价不意味着 bit-level 等价。

从这个角度看，数学层保留不变量与收敛条件，调度层负责无冲突 round，layout 层在行主序契约与列主序工作区之间切换， kernel 层暴露 block 级规约与内存访问模式，主机层维护资源生命周期、自动调优与全局同步。这样的分层使实现既不背叛算法语义，也不假装硬件约束不存在。

3.4 主机侧契约：RAII 管资源，kernel 管计算

`kernels.cu` 的系统边界不是具体 kernel，而是资源所有权（resource ownership）边界。设备矩阵由 `DeviceMatrix` 封装：

$$\texttt{DeviceMatrix} = (\texttt{rows}, \texttt{columns}, \texttt{double* data}).$$

它负责 `cudaMalloc`、`cudaFree`、主机到设备拷贝（host-to-device copy）以及设备到主机拷贝（device-to-host copy）。拷贝构造与拷贝赋值被显式禁用，移动构造与移动赋值被允许。这是资源获取即初始化（Resource Acquisition Is Initialization, RAII）策略：对象生命周期与 GPU 内存生命周期保持一致。

这一边界对于 CUDA 程序尤为关键。kernel 不应携带复杂的 C++ 对象语义，也不应参与资源所有权决策。它只接收

$$\texttt{double*},\quad \texttt{const double*},\quad \texttt{int},\quad \texttt{int2*}$$

等平坦参数。换言之：

> 主机侧对象维护资源正确性；设备侧函数执行数值变换。所有权、异常处理、安全检查与数学算子应当保持清晰分层。

这种划分降低了系统边界处的复杂度。实际 CUDA 程序中的错误常来自职责不清：谁释放内存，谁检查错误，谁维护矩阵维度，谁负责同步。当前实现将这些职责分离： `DeviceMatrix` 管理矩阵资源，匿名命名空间中的 `DeviceBuffer<T>` 管理临时数组， `JACOBI_CUDA_CHECK` 将 CUDA 错误码转换为异常，而 kernel 只处理裸数据上的数值计算。

公开入口 `one_sided_jacobi_svd` 因此呈现出近似纯函数式外观：

$$(A, C) \mapsto (U, \Sigma, V, s),$$

其中 A 是主机侧行主序输入矩阵，C 是类型化计算配置，s 是实际 sweep 数。内部可以执行 CUDA 分配、主机与设备之间的拷贝、kernel launch、布局转置和同步；外部得到的是主机侧值对象，而不是 device pointer 或临时 workspace。

这种外观不是美学装饰，而是系统边界纪律。kernel 不认识文件格式、路径、线程池、输出顺序或 CLI 字符串；它只认识矩阵、配置和数值结果。下一章会继续把这条边界扩展到系统层： I/O 负责把外部字节变成矩阵，pipeline 负责把矩阵流组织成用例，kernel 只执行单矩阵 SVD。

3.5 内存布局、转置策略与自动调优

kernels.cu 没有把外部行主序契约强行等同于内部工作布局。它同时提供行主序索引

$$\texttt{row_major_index}(i, j, n) = i \cdot n + j$$

与列主序（column-major）索引

$$\texttt{column_major_index}(i, j, m) = j \cdot m + i,$$

并用模板策略 matrix_index<ColumnMajor> 将两条路径统一到同一组统计与旋转 kernel 中。这使外部应用二进制接口（Application Binary Interface, ABI）风格的数据布局保持稳定，内部则可以根据硬件访存代价选择更合适的表示。

当前版本采取双路径（dual-path）策略：

- **直接路径。** 工作矩阵 A 保持行主序，统计核与旋转核以 ColumnMajorA=false 实例化，因而使用跨步列访问。
- **布局转置路径。** 输入先从行主序重排到列主序工作区 d_a_layout，统计核与旋转核以 ColumnMajorA=true 实例化，收敛后再转回行主序，以便 build_u_sigma_kernel 和主机侧输出继续使用统一格式。
- **V 的固定策略。** 右奇异矩阵 V 始终保持行主序。它的规模是 $n \times n$，且更新成本通常小于 $m \gg n$ 时工作矩阵 A 的列统计与列更新成本；把布局复杂度集中在 A 上，可以避免优化扩散到整个系统。

布局转置由 transpose_row_major_kernel 完成。它使用 32×32 的共享内存 tile，并在共享内存第二维加一列填充：

$$\texttt{tile[32][33]}.$$

这个 +1 用于缓解共享内存存储体冲突（bank conflict）。读入阶段和写出阶段尽量让全局内存访问保持连续，而转置所需的非连续重排被限制在 shared memory tile 内部。

JacobiSvdConfig 将布局选择显式暴露为 LayoutTransposeMode：

$$\texttt{auto_select}, \quad \texttt{force_enable}, \quad \texttt{force_disable}.$$

强制启用与强制禁用主要服务于验证和 benchmark；默认 auto_select 使用两个阈值判断是否启用布局转置：

$$n \geq \texttt{layout_transpose_min_columns}, \qquad mn \geq \texttt{layout_transpose_min_elements}.$$

默认值分别是 16 与 4096。这两个阈值表达一个朴素判断：矩阵列数太小或元素总量太小的时候，转置 kernel、额外设备缓冲区与同步边界的固定成本可能吞掉连续列访问带来的收益。

自动调优（auto-tuning）进一步把该判断从静态经验值变成运行时微基准。若 pipeline 配置中 layout_transpose_auto_tune=true 且模式仍为 auto_select，应用层会在正式处理 test-case 前调用

$$\texttt{auto_tune_layout_transpose_threshold}.$$

该函数扫描一组代表性列数：

$$\{8, 12, 16, 24, 32, 48, 64, 96\},$$

并使用 $m = 2n$ 的确定性基准矩阵分别测量直接路径与布局转置路径。调优报告对象记录：

`recommended_min_columns`, `recommended_min_elements`,

`estimated_best_speedup`, `sample_count`.

pipeline 只把推荐阈值写入本次运行的 kernel 配置，随后关闭该配置中的自动调优开关，避免每个 testcase 重复调优。它不修改全局状态，也不把调优结果持久化到磁盘。

这个设计仍有边界。微基准只覆盖固定形状族 $m = 2n$，给出的是对当前 GPU、CUDA runtime 与执行配置的经验建议，不是跨机器、跨 workload 的数学定理。若生产输入的 m/n 比例、矩阵数量或 sweep 分布明显不同，用户仍应使用强制开关与显式阈值做对照实验。

3.6 从列对到无冲突 round

串行单边雅可比会遍历所有列对：

$$(p, q), \qquad 0 \le p < q < n.$$

但在 GPU 上，如果两个线程块同时更新 (p, q) 与 (p, r)，它们都会写第 p 列，结果将产生数据竞争（data race）。因此，并行化的第一步不是盲目增加线程数量，而是构造无冲突调度。

当前实现使用巡回赛排序（round-robin / tournament ordering）。每个 round 包含若干互不共享列的列对：

$$(p_1, q_1), (p_2, q_2), \ldots, (p_k, q_k), \qquad \{p_i, q_i\} \cap \{p_j, q_j\} = \varnothing \quad (i \neq j).$$

同一 round 内的列对可以并行执行；不同 round 之间必须顺序执行，因为后一个 round 依赖前一个 round 更新后的列。

若 n 为奇数，实现会加入一个 dummy 列参与轮转；凡是匹配到 dummy 的列对会被跳过。这样可以把奇偶差异纳入普通调度逻辑，而不是在多个 kernel 中扩散特判。这是比较“好品味”的处理：特殊情况不是消失了，而是被压缩到一个清晰的调度生成层。

3.7 规约：局部 Gram 矩阵的并行化

规约是 GPU 编程中最容易受到串行直觉误导的算子之一。串行代码中的点积通常写成：

$$s = 0; \qquad s \leftarrow s + x_i y_i.$$

该写法隐含了“单一累加器顺序遍历”的模型。但在 GPU 上，若由单个线程遍历完整列，绝大多数线程都会闲置；若让所有线程同时写同一个累加器，又会引入原子热点与不可控的累加顺序。更合适的模型是树形合并：每个线程先维护私有局部累加器，再把局部结果按层次结构合并。

当前 `pair_stats_kernel` 对每个列对一次性规约三个量：

$$\alpha = a_p^T a_p, \qquad \beta = a_q^T a_q, \qquad \gamma = a_p^T a_q.$$

网格组织方式是“一列对一个 block”。block 内所有线程分摊行维度：

```
for row = tid; row < m; row += blockDim.x.
```

每个线程先计算局部和：

$$\alpha_{\text{local}}, \quad \beta_{\text{local}}, \quad \gamma_{\text{local}},$$

再写入共享内存（shared memory）。共享内存被划分为三段：

$$\texttt{shared_pp}, \qquad \texttt{shared_qq}, \qquad \texttt{shared_pq}.$$

随后通过 `__syncthreads()` 保证所有局部和对整个 block 可见，并执行二分树形规约：

$$\texttt{stride} = \frac{B}{2}, \frac{B}{4}, \ldots, 1.$$

最终，三个共享内存数组的 0 号槽位就是该列对的规约结果。

该规约方式有三个需要显式记录的性质：

- 它隐含线程数为 2 的幂这一良性条件。默认 `threads_per_block=256` 是安全配置；若允许任意 warp 对齐值，例如 384，当前二分规约可能遗漏槽位，因此后续应收紧配置约束或修改规约逻辑。
- 它没有使用 warp-level shuffle。共享内存规约更直观、可移植性更好；但现代 GPU 上可通过 `__shfl_down_sync` 降低共享内存压力与同步成本。
- 它的求和顺序不同于串行循环。若需要更高精度，可以考虑 pairwise summation 或 Kahan summation，但这会增加寄存器占用与指令开销。

收敛后的 `build_u_sigma_kernel` 采用同一模式：一列一个 block，先规约列范数

$$\sigma_j = \sqrt{\sum_i a_{ij}^2},$$

再将该列归一化为

$$u_j = a_j / \sigma_j.$$

这里，规约不仅是性能技巧，也是算法语义的一部分：奇异值本身即由收敛后工作矩阵的列范数定义。

3.8 kernel 管线与同步边界

一次 sweep 由若干 round 组成。布局转置路径若被启用，会在 sweep 循环前后各执行一次转置；但每个 round 的数值核心稳定地拆成三类 kernel：

这里的“稳定”很重要。布局策略没有复制一套完整算法，而是通过统计核与旋转核的 `ColumnMajorA` 模板参数切换 A 的索引方式。因此，直接路径与转置路径共享同一套 round-robin 调度、旋转参数计算和收敛判断。特殊情况没有扩散到算法主循环里，复杂度被约束在布局策略层。

阶段	kernel	职责
统计	`pair_stats_kernel`	为每个列对计算 α, β, γ
参数	`compute_rotation_params_kernel`	根据局部 Gram 矩阵计算 c, s
应用	`apply_rotation_kernel`	把旋转同时应用到 A 与 V

`compute_rotation_params_kernel` 读取统计量，并使用稳定的 Rutishauser 公式：

$$\tau = \frac{\beta - \alpha}{2\gamma}, \qquad t = \begin{cases} \dfrac{1}{\tau + \sqrt{1+\tau^2}}, & \tau \geq 0, \\ -\dfrac{1}{-\tau + \sqrt{1+\tau^2}}, & \tau < 0, \end{cases}$$

$$c = \frac{1}{\sqrt{1+t^2}}, \qquad s = tc.$$

如果

$$|\gamma| \leq \varepsilon\sqrt{\alpha\beta},$$

则认为该列对已经足够正交，并令 $c = 1, s = 0$。为避免极端下溢附近的除法风险，代码还要求

$$|\gamma| > 10^{-300}.$$

只要某个列对需要旋转，就使用 `atomicExch` 将 `any_rotation_flag` 置为 1。该 flag 构成 sweep 收敛判断的桥梁：设备端记录“仍存在需要旋转的列对”，主机端在 round 结束后将该状态拷回 CPU。

`apply_rotation_kernel` 同样以“一列对一个 block”的方式工作。它把同一组 (c, s) 同时应用到 A 的两列与 V 的两列：

$$a'_p = ca_p - sa_q, \qquad a'_q = sa_p + ca_q,$$

$$v'_p = cv_p - sv_q, \qquad v'_q = sv_p + cv_q.$$

同步更新 V 是必要的。若只更新 A 而不更新 V，实现只能得到正交化后的列和奇异值，却无法恢复右奇异向量。当前实现维护的不变量仍然是

$$A^{(k)} = A^{(0)}V^{(k)}.$$

这份实现的同步层次较为清晰：

- block 内同步由 `__syncthreads()` 完成，用于共享内存规约；
- layout 转置路径在 sweep 前后各有一个 kernel launch 边界；
- kernel launch 边界提供全局同步，保证统计、参数计算与旋转应用按顺序发生；
- round 之间由主机循环顺序化，避免不同 round 的列更新交叉；
- sweep 收敛由 `any_rotation_flag` 从设备拷回主机后判断。

这不是最激进的性能路径，但它把全局同步放在清晰的 kernel 边界，而不是试图在单个巨大 kernel 中模拟跨 block 同步。普通 CUDA kernel 不提供任意 block 之间的同步原语；若强行模拟，实现会显著复杂化且更难验证。以工程稳健性为优先时，清晰的 kernel launch 边界通常优于隐式同步技巧。

3.9 执行配置与资源占用

当前实现把列对作为并行粒度：

$$\text{one column pair} \longleftrightarrow \text{one CUDA block.}$$

block 内线程沿行维度分摊工作。若线程数为 T，矩阵行数为 m，则每个线程大致处理

$$\left\lceil \frac{m}{T} \right\rceil$$

个元素。随后，所有线程把局部累加结果写入 shared memory，并通过树形规约得到该列对的统计量。

执行配置（execution configuration）至少影响四件事：

- **并行度。** T 越大，单个列对内部的行维度并行度越高；但当 $T > m$ 时，大量线程没有有效元素可处理。
- **共享内存占用。** `pair_stats_kernel` 为每个 block 维护三段共享内存，共

  $$3T \cdot \text{sizeof}(\texttt{double})$$

  字节。默认 $T = 256$ 时约为 6144 字节，通常不会成为主要瓶颈。
- **occupancy。** 占用率（occupancy）取决于每个 block 的线程数、寄存器数量（register count）、shared memory 和设备的最大驻留 block 数。高 occupancy 能帮助隐藏 global memory 延迟，但不自动等价于高吞吐；若访存模式高度跨步，更多 warp 也只是更努力地等待内存。
- **规约正确性。** 当前共享内存二分规约隐含 T 为 2 的幂这一条件。因此执行配置不只是性能旋钮，它也参与正确性契约。

这揭示了 GPU 程序的一个常见陷阱：`threads_per_block` 看起来像“调大就更快”的旋钮，实际上同时改变并行粒度、资源驻留、同步成本和规约结构。对于本项目，保守默认值比激进调参更重要： SVD kernel 的首要目标是稳定地产生可验证结果；性能调优必须建立在这个稳定点之上。

若未来把执行配置提升为严肃 benchmark 参数，建议采用：

$$T \in \{64, 128, 256, 512\},$$

只允许 2 的幂，并在报告中记录 T、GPU 型号、CUDA 版本与矩阵形状。否则，不同实验之间的性能差异可能来自执行配置漂移，而不是算法实现本身。

3.10 当前实现的边界与后续方向

当前 `one_sided_jacobi_svd` 明确要求

$$m \geq n.$$

这与 thin SVD 的常见场景一致：列数不超过行数，输出 $U \in \mathbb{R}^{m\times n}$、$\Sigma \in \mathbb{R}^n$、$V \in \mathbb{R}^{n\times n}$。若要支持宽矩阵（wide matrix）$m < n$，通常可以对 A^T 执行相应分解，或重新设计输出形状与后处理逻辑。

另一个边界是收敛判断。当前实现每个 round 都将 `any_rotation_flag` 拷回主机。该方式直观且易于验证，但会引入频繁的小规模 host-device 同步。后续优化可以将 flag 延迟到每个 sweep 结束后再汇总，或使用设备端数组记录所有 round 状态，以减少 CPU 与 GPU 之间的往返。

布局转置已经缓解了行主序与列规约之间最直接的冲突，但它不是免费的午餐。当前自动调优只扫描有限尺寸集合，并假设基准矩阵形状为 $m = 2n$；它适合给出本次运行的阈值建议，不适合被解读为跨 workload 的最优策略。

若未来目标从“清晰正确”转向“吞吐最大化”，更值得继续研究的方向是：

- 使用 tile 将列对局部缓存到 shared memory，减少重复 global memory 访问；
- 使用 warp-level reduction 减少共享内存访问和 `__syncthreads()` 次数；
- 降低 host-device flag 往返频率，减少小规模同步对流水的破坏；
- 扩展自动调优样本，使其覆盖更多 m/n 比例与执行配置；
- 为 benchmark 报告补充 GPU 型号、CUDA 版本、线程配置、布局策略与验证残差。

这些方向的共同点是：它们优化的不是 SVD 的数学公式，而是公式在机器上的数据运动路径。高性能计算里经常有这样反直觉的事实：真正昂贵的不是计算本身，而是把正确的数据在正确时间送到正确执行单元。本章的核心设计感也在这里：先用数值不变量定义“什么叫对”，再用 roofline 与体系结构映射解释“为什么这样实现”，再固定对象契约与内存布局，最后展开 round、规约、kernel 管线与同步。这样，CUDA 代码就不只是实现细节，而是数学模型进入机器世界时留下的清晰轨迹。

4 从 Kernel 到系统：纯函数构件与良构边界

上一章已经把 `kernels.cu` 的数值核心展开到 CUDA 执行结构上：单边雅可比（one-sided Jacobi）把列对组织成无冲突 round，block 内规约（reduction）计算局部 Gram 信息，kernel launch 边界承担 round、统计、旋转与后处理之间的全局同步。这些内容回答的是一个局部而硬核的问题：*如何让 GPU 正确执行 SVD 的核心变换*。

本章换一个视角。一个可运行、可维护、可复现实验的数值系统，还必须回答另一个问题：*这个 kernel 应该以什么形态嵌入系统*。如果 GPU kernel 直接认识文件格式、路径、命令行参

数、输出队列、线程池和构建目录，那么最昂贵、最难调试的一层就会被迫承担最多变化。这不是高性能，这是把复杂度塞进最脆弱的位置。

本章的核心论点是：CUDA kernel 应当被设计成系统中的纯函数式构件（pure-function-like component），而不是应用程序本身。严格地说，CUDA kernel 会写设备内存、依赖 runtime 调度、消耗 GPU 资源，并不满足数学意义上的纯函数（pure function）。但是在系统边界上，它应当表现得像一个确定的映射：

$$\mathrm{svd}_{\mathrm{cuda}} : (A, C) \longmapsto (U, \Sigma, V, r).$$

其中 A 是主机侧行主序输入矩阵，C 是类型化计算配置，U, Σ, V 是 SVD 输出，r 是当前 kernel 层能够报告的计算摘要。在当前实现中，这个摘要具体表现为 `JacobiSvdResult::sweeps`；更完整的端到端执行报告，例如 testcase 数、输出矩阵数、布局转置自动调优结果，由应用层 `PipelineReport` 汇总，而不是塞进 kernel 的语义内部。

这个区分很重要。kernel 可以返回“我对这张矩阵迭代了多少 sweep”；pipeline 才能回答“本次输入流一共处理了多少 testcase、写出了多少矩阵、运行时阈值是多少”。前者是单矩阵数值事实，后者是端到端系统事实。把这两类事实混在一起，会让接口表面看起来更方便，实际却更难测试、更难替换、更难解释。

4.1 边界的必要性：高性能代码最怕职责膨胀

GPU kernel 同时承受三类复杂性。

- **数学复杂性**：单边雅可比需要维护列正交性、收敛阈值、奇异值提取、左右奇异向量关系以及 $m \geq n$ 的 thin SVD 前置条件；
- **体系结构复杂性**：CUDA 的 thread、block、warp、shared memory、 global memory、kernel launch 与 host-device 同步都会影响正确性和性能；
- **资源复杂性**：设备内存分配、主机到设备拷贝、设备到主机拷贝、错误码翻译、同步时机以及临时缓冲区生命周期必须被严格管理。

如果再让 kernel 层理解 `*.mat` 头部、文本矩阵空行、输出路径、CLI 默认值、线程池窗口和 JSON 风格报告，它就会变成所有变化的交汇点。这样的设计在小 demo 里看似直接，在真实演化中会迅速失控：文件格式变化影响 kernel，命令行变化影响 kernel，输出顺序变化影响 kernel，最终最难验证的一层承担了最多责任。

这就是反向依赖（reverse dependency）的坏味道：越接近数学核心的代码，越不应该认识越外层的世界。为了让边界足够锋利，需要把 kernel 不该知道的东西说清楚：

- **不认识文件格式。** `kernels.cu` 不解析 `*.mat` 的网络字节序头部，也不解析 `*.txt` 中的空格、换行和空行；
- **不认识路径与构建产物。** 输入路径、输出路径、父目录创建、CMake 生成器、 `nvcc` driver mode 与 artifact 位置都不应影响 SVD 语义；

- **不认识用户意图。** `--epsilon`、`--max-sweeps` 等命令行拼写在进入 kernel 前应已变成类型化配置；
- **不认识端到端调度。** testcase index、线程池（thread pool）、future 窗口、重排序缓冲区（reorder buffer）、完成队列和输出顺序都属于应用层；
- **不拥有长期系统资源。** 文件句柄、映射视图、输出线程、完成队列和 CLI 报告格式都不属于 kernel。

这组否定边界不是装饰性的架构洁癖，而是在保护高性能代码最稀缺的东西：可定位性。kernel 的尊严来自克制。它不处理世界，它只处理矩阵。

良构边界（well-formed boundary）的目标不是把代码切成漂亮图层，而是降低变化传播（change propagation）。理想情况下：

$$\Delta\text{CLI} \nrightarrow \Delta\text{Kernel}, \qquad \Delta\text{File Format} \nrightarrow \Delta\text{Jacobi Rotation},$$

$$\Delta\text{Output Ordering} \nrightarrow \Delta\text{CUDA Memory Layout}, \qquad \Delta\text{Build Artifact} \nrightarrow \Delta\text{SVD Semantics}.$$

这意味着用户接口、持久化格式、线程调度与构建方式都可以独立演化，而不会倒灌进数值核心。对 CUDA 数值程序来说，这不是学院派洁癖，而是长期调试成本的生命线。

4.2 轻量 DDD：领域不是业务，而是数值不变量

领域驱动设计（Domain-Driven Design, DDD）常见于复杂业务系统，例如订单、账户与库存。若机械套用，把每个类都命名成实体（Entity）、值对象（Value Object）或仓储（Repository），这个项目会立刻沾上不必要的企业应用气味。这里真正有用的不是术语，而是 DDD 的朴素核心：*系统应围绕领域不变量组织，而不是围绕外部技术细节组织。*

在本项目中，“领域”（domain）不是业务规则，而是 SVD 的数值语义：

$$A \approx U\Sigma V^T, \qquad U^TU \approx I, \qquad V^TV \approx I,$$

以及单边雅可比迭代中的局部不变量：

$$a_p^T a_q \to 0, \qquad \|a_p\|_2, \|a_q\|_2 \text{ 由正交旋转稳定更新}.$$

这些关系才是系统真正要保护的核心。文件扩展名、命令行拼写、队列容量、CMake 生成器只是让这个核心进入现实世界的外层机制。

因此，本项目采用的是轻量 DDD：只保留“领域核心、应用编排、基础设施、表示层”这些边界思想，不强行引入完整战术模式。它真正关心的是依赖方向（dependency direction）：

$$\texttt{main.cpp} \longrightarrow \texttt{pipeline.cpp} \longrightarrow \{\texttt{io.cu}, \texttt{kernels.cu}\}.$$

也就是说，表示层（presentation layer）把用户字符串编译成配置；应用层（application layer）把一次用例编排成读入、计算、写出；基础设施层（infrastructure layer）把外部字节变成矩阵流；领域核心（domain core）只执行单矩阵 SVD 数值变换。外层知道内层，内层不反向知道外层。`io.cu` 与 `kernels.cu` 在应用层之下并列：前者负责数据进入系统，后者负责数值核心

计算，二者不应相互依赖。kernel 不读取文件；I/O 不执行 SVD； pipeline 才是把它们组合成用例的地方。

把这个方向压缩成公式，就是：

$$\text{System} = \text{Adapters} \circ \text{Application Use Case} \circ \text{Domain Kernel}.$$

这里的 $\circ$ 不是单纯函数复合，而是责任边界的复合：每层只把下一层需要的最小信息传过去。

围绕这个方向，良构系统应长期保持几条不变量（invariant）：

- 领域核心只依赖内存中的类型化数据，不接收路径、文件句柄、原始字节流或 CLI 字符串；
- 公开结果不暴露设备资源，`JacobiSvdResult` 不返回 device pointer、stream、 event 或临时缓冲区；
- 应用层只编排用例，不复制 Jacobi 旋转公式，也不重写 SVD 后处理逻辑；
- 基础设施层可以选择 `*.mat`、文本、mmap 或 pinned host buffer，但不能改变 ϵ、最大 sweep 数、列对调度或奇异值语义；
- 表示层只能生成配置并调用 pipeline，不能为了方便直接读文件、写矩阵或调用内部 kernel helper；
- 错误在边界被翻译，而不是在深处格式化给用户；
- 并发不改变输出契约，线程池可以让 testcase 乱序完成，但输出文件仍必须按输入顺序写出；
- 构建 artifact 不改变运行语义，同一输入、同一配置和同一源码应暴露相同应用层契约。

这些不变量把 DDD 从“分层名词表”变成了可检查的工程纪律。换句话说，小可爱 Klee，这里的 DDD 不是给代码穿西装，而是给数值核心画防火线。

4.3 Kernel 的公开契约：窄入口，类型化配置，主机侧结果

当前 kernel 层向外暴露的核心入口是：

$$\texttt{one_sided_jacobi_svd}: \quad (\texttt{host_input}, rows, columns, C) \longrightarrow \texttt{JacobiSvdResult}.$$

这个接口有三个值得保留的性质。

第一，输入是内存中的类型化矩阵，而不是外部世界。 kernel 入口只接收主机侧行主序（row-major）数值数组、行列数和配置。它不接收路径、不接收文件句柄、不接收网络序字节流，也不接收 `argv`。坏文件必须在 I/O 层被拒绝，坏命令行必须在 CLI 层被拒绝；kernel 只处理已经成为矩阵的数据。

第二，配置显式表达数值与执行策略。 `JacobiSvdConfig` 目前包含：

$$\epsilon, \quad \texttt{max_sweeps}, \quad \texttt{threads_per_block},$$

以及布局转置相关策略：

$$\texttt{layout_transpose_mode}, \quad \texttt{layout_transpose_min_columns},$$
$$\texttt{layout_transpose_min_elements}$$

$$\texttt{layout_transpose_auto_tune},$$
$$\texttt{layout_transpose_benchmark_repetitions},$$
$$\texttt{layout_transpose_benchmark_sweeps}.$$

这些字段并不全是同一种语义。ϵ 与 `max_sweeps` 更接近数值算法参数； `threads_per_block` 与布局转置阈值更接近执行策略（execution policy）；自动调优字段则描述“如何在本次运行前测量阈值”。当前把它们放在同一个结构体中是可接受的，因为项目规模还小；但文档必须明确这种混合，以免未来把所有性能遥测和设备状态都继续塞进去。

第三，输出是主机侧值对象，而不是设备指针。 `JacobiSvdResult` 包含：

$$rows, \quad columns, \quad u, \quad sigma, \quad v, \quad sweeps.$$

其中 U 为 $m \times n$ 行主序矩阵，Σ 为长度 n 的数组， V 为 $n \times n$ 行主序矩阵。设备指针、临时统计数组、旋转参数数组和布局工作区都不逃出 `kernels.cu` 的边界。外部世界拿到的是可移动、可测试、可序列化的 C++ 值。

这就是“纯函数式外观”的核心：内部可以有 `cudaMalloc`、`cudaMemcpy`、 kernel launch 和 `cudaDeviceSynchronize`；外部看到的是

$$(A, C) \mapsto (U, \Sigma, V, s).$$

副作用被收束在窄边界内，接口语义因此接近数学函数。

4.4 外层支撑：让纯 kernel 落到真实系统

kernel 的契约只覆盖单张矩阵的数值变换，但程序要跑起来，还需要外层把“脏世界”整理成 kernel 能接受的形状。表示层、应用层和基础设施层的共同价值，不是替 kernel 做数学，而是给 kernel 提供干净输入、稳定调度、可解释输出和受控资源。

$$argv \longrightarrow \texttt{PipelineConfig} \longrightarrow \text{matrix stream}$$
$$\longrightarrow \texttt{JacobiSvdResult} \longrightarrow \text{output stream}.$$

`main.cpp` 承担意图编译器（intent compiler）的角色。它把命令行字符串解析成 pipeline 配置和 kernel 配置，并在进入应用层前完成默认值归一化与基本验证。它的产物应该是类型化配置，而不是一次偷偷的文件读取或 kernel launch。

`io.cu` 承担输入输出适配器（I/O adapter）的角色：它把 `*.mat`、文本格式、字节序、内存映射和页锁定主机缓冲区（pinned host buffer）封装在边界内，向上交付 `io::Matrix` 或带

序号的派发任务。它可以让数据更适合高吞吐流动，但不能决定 SVD 的数值语义，也不能把文件映射地址直接塞给 kernel。

`pipeline.cpp` 承担用例协议（use-case protocol）的角色：它把输入、计算、输出组合成一次端到端执行，负责 testcase 序号、线程池（thread pool）、future 窗口、重排序缓冲区（reorder buffer）、背压（backpressure）和跨线程异常传播。它可以让多个 testcase 并发完成，但必须保持输出契约：

$$(U_0, \Sigma_0, V_0), (U_1, \Sigma_1, V_1), \ldots$$

仍按输入顺序写出。

因此，外层三件事合起来给 kernel 提供的是一个降噪通道：

$$\text{external bytes and user strings} \xrightarrow{\text{parse, validate, schedule}} \text{typed host matrices and configs}$$
$$\xrightarrow{\text{kernel}} \text{typed numerical results} \xrightarrow{\text{order, encode, report}} \text{user-visible artifacts.}$$

这就是它们在系统设计中的职能：把现实世界的不稳定性吸收在外层，让 kernel 面对尽可能稳定、尽可能窄的数学接口。

4.5 健壮性要求：所有权清楚，失败可翻译

边界要真正可依赖，只靠“分层”不够；资源所有权和错误语义也必须落到代码上。资源获取即初始化（Resource Acquisition Is Initialization, RAII）在这里不是 C++ 技巧，而是系统健壮性（robustness）的基本要求：谁申请资源，谁负责释放；资源生命周期不能漂移到不该认识它的层。

当前系统中，设备矩阵对象拥有设备内存；读写器拥有文件映射和句柄； pinned buffer 拥有页锁定主机内存；输出阶段和全局线程池拥有线程、队列与关闭协议。每个资源都应满足唯一所有者（unique owner）关系：

$$\text{owner}(r) = o, \qquad \text{lifetime}(r) \subseteq \text{lifetime}(o).$$

这条纪律消除的是 CUDA/C++ 混合程序里最毒的故障：重复释放、早释放、异常路径泄漏、线程未 join、映射视图悬垂、device pointer 漂移到上层。

错误边界同样要清楚。系统错误可以粗分为：

$$E = E_{\text{cli}} \cup E_{\text{io}} \cup E_{\text{kernel}} \cup E_{\text{pipeline}} \cup E_{\text{build}}.$$

CLI 参数错误应在运行前报告；I/O 格式错误应指出记录边界或维度问题； CUDA runtime 错误应保留底层诊断；pipeline 错误应说明并发阶段、输出缺包或线程异常；构建错误应停留在 CMake 或编译器层。

kernel 层尤其需要错误翻译（error translation）。CUDA API 返回 `cudaError_t`，这是 runtime 语言；应用层更需要知道的是“哪次设备操作失败、失败码是什么、发生在何处”。`JACOBI_CUDA_CHECK` 与 `CudaError` 的价值就在这里：把底层错误提升为稳定异常语义，同时保留表达式、文件、行号和 CUDA 错误字符串。

还要区分两类常被混淆的失败：

$$\text{algorithmic non-convergence} \neq \text{CUDA runtime failure.}$$

达到 `max_sweeps` 仍未完全收敛，是数值算法状态；`cudaMalloc`、 kernel launch 或 `cudaMemcpy` 失败，是运行时系统状态。前者适合进入计算报告或验证指标，后者通常应终止当前任务并传播诊断。pipeline 再把 worker 异常、写线程异常和关闭路径异常汇总回 `JacobiSvdPipeline::run()`。只要任一阶段认为本次输出不可信，就不能报告成功。

4.6 实体生命周期：同名“矩阵”不是同一种东西

如果只看名字，系统里到处都是“矩阵”；但从实体生命周期（entity lifecycle）的角度看，这些“矩阵”根本不是同一种东西。它们有不同的出生位置、不同的合法操作、不同的所有者，也有不同的死亡方式。把它们统一叫作 matrix 没问题；把它们当成同一种实体来传递、缓存、复用和暴露，就会立刻引入边界污染。

从数据结构角度看，系统里至少存在六种不同层次的“矩阵”：

- 文件中的矩阵：网络字节序头部、载荷和持久化布局；
- 派发任务中的矩阵：带序号的原始 payload 与 pinned host buffer；
- 主机侧矩阵：`io::Matrix`，即维度加行主序 `std::vector`；
- 设备侧矩阵：`DeviceMatrix`，即维度加 `double*`；
- 内部工作矩阵：可选的列主序 `d_a_layout`；
- 输出矩阵：按 U、$1 \times n$ 的 Σ、V 三张矩阵写入流。

更准确地说，它们对应同一个数值对象在系统里的六个生命阶段：

$$\text{persisted} \to \text{dispatched} \to \text{typed on host}$$
$$\to \text{resident on device} \to \text{workspace-transformed} \to \text{materialized as outputs.}$$

这个视角的重要性在于：*边界不是按“长得像不像矩阵”来切，而是按“此刻它处于哪段生命周期”来切。*

第一段：持久化实体（persisted entity）。 文件中的矩阵还不是数值计算对象，而是外部表示（external representation）。它的职责是可保存、可传输、可回放，因此它关心的是字节序、记录边界、文件完整性与载荷布局。这一阶段的“矩阵”归 I/O 层所有，最典型的操作是 decode，而不是 SVD。

第二段：派发实体（dispatched entity）。 当主线程用单游标扫描输入流时，文件中的一段 payload 被包装成带 testcase 序号的任务，再附带 pinned host buffer 或延迟解析信息，准备交给 worker。这里的矩阵已经进入系统，但它仍然首先是调度对象（scheduling object），不是领域对象。它携带顺序、来源、缓冲区占用和背压关系，服务的是 pipeline，而不是 Jacobi 旋转。

第三段：主机侧类型化实体（typed host entity）。 `io::Matrix` 才是第一个真正适合拿来谈“行数、列数、row-major 数组”的矩阵对象。到这一步，原始字节已经被解释为类型化数据，应用层和测试代码终于可以在不关心文件格式的前提下讨论数值正确性。它适合做边界输入、单元测试和结果比对，但它仍不该承担设备资源管理。

第四段：设备驻留实体（resident device entity）。 `DeviceMatrix` 是计算载体，不是系统通用值对象。它的核心属性不是“也是矩阵”，而是“它的存活依赖 CUDA 上下文、分配器和释放时机”。一旦这个对象越过 kernel wrapper 泄漏到 pipeline，外层就不得不理解 `cudaMalloc`/`cudaFree`、同步点和异常路径，边界会立刻反向塌陷。

第五段：内部工作实体（workspace entity）。 列主序工作区 `d_a_layout` 更进一步，它甚至不是公开语义里的输入或输出矩阵，而只是某种执行策略下的中间形态。它为了访存模式（memory access pattern）和布局转置存在，生命周期应严格限制在一次 kernel 调用或一次 SVD 计算内部。若这种工作区被提升为公共接口的一部分，就等于把“某次优化是否值得启用”错误地升级成“整个系统都必须理解的概念”。

第六段：结果实体（materialized result entity）。 输出阶段重新得到的是主机侧可序列化结果：U、Σ、V 三者作为数值产物，再被编码回输出流。注意这已经不是“输入矩阵的继续存在”，而是一次计算完成后物化（materialization）出来的新实体。它们可以被写盘、校验、比较、统计 sweeps，但不应再倒流回设备工作区语义。

因此，它们不能混为一谈。文件矩阵适合 I/O 层，派发任务适合 pipeline 的输入边界，`io::Matrix` 适合应用层和测试；`DeviceMatrix` 适合 kernel wrapper；列主序工作区只属于 `kernels.cu` 内部。若把设备指针暴露给 pipeline，应用层就会开始关心 `cudaFree`；若把文件映射地址暴露给 kernel，数值核心就会开始关心平台页缓存。两种方向都是坏味道。

从“微操”角度看，这个 subsection 真正想保护的是一句很朴素的话：*同一份数学对象，在不同生命周期阶段，应当由不同数据结构承载。* 不这样做，后果通常不是抽象上“不优雅”，而是非常具体的工程事故：重复拷贝、悬垂指针、异常路径泄漏、测试夹杂平台细节、性能优化打穿语义边界，以及外层代码悄悄依赖内部工作区布局。

正确的数据流是逐层降噪：

$$\text{file bytes} \xrightarrow{\text{decode or dispatch}} \text{host matrix} \xrightarrow{\text{copy}} \text{device matrix}$$
$$\xrightarrow{\text{compute}} \text{host result} \xrightarrow{\text{encode}} \text{file bytes.}$$

其中 `*.mat` 高吞吐路径把第一段细化为：

$$\text{file bytes} \xrightarrow{\text{single cursor}} \texttt{MatDispatchTask} \xrightarrow{\text{worker decode}} \texttt{io::Matrix}.$$

如果进一步压缩成生命周期语言，那么它表达的是：

$$\text{born outside} \to \text{typed inside} \to \text{computed privately} \to \text{published again.}$$

复杂性没有消失，但它被放到了可定位的位置；资源也没有消失，但每种资源只在自己那一段生命里存活。

4.7 当前实现的边界风险

当前实现已经体现了上述方向，但仍有几处需要持续警惕。

第一，`JacobiSvdConfig` 同时包含数值参数与执行策略。 这在当前规模下可以接受，但未来若加入 device id、stream policy、workspace 预算、profiling 开关，最好区分数学配置、执行配置和实验观测配置。否则配置对象会变成“所有旋钮的大口袋”。

第二，kernel 层仍是同步接口。 `one_sided_jacobi_svd` 当前是同步风格：调用完成后结果已回到主机。 pipeline 可以跨 testcase 并发提交 host 任务，但这不等价于显式 CUDA stream 级流水线。若未来要实现 host decode、host-device copy、kernel、device-host copy 的真正重叠，需要新增异步 compute port，而不是在 pipeline 里偷偷管理 stream 生命周期。

第三，执行配置仍有正确性含义。 `threads_per_block` 看似只是性能参数，但上一章已经指出，当前共享内存二分规约对线程数形态有隐含假设。默认 256 是安全点；后续最好把合法值收紧为 2 的幂，或修改规约以正确处理任意 warp 对齐线程数。性能旋钮不能悄悄参与正确性。

第四，结果报告仍较粗。 kernel 只报告 `sweeps`；pipeline 报告 testcase 数、写出矩阵数、总 sweep 与布局调优摘要。这足以表达语义成功，但还不是性能遥测（performance telemetry）。若要定位吞吐瓶颈，应新增独立 profiling 结构，记录阶段耗时、队列等待、重排序峰值与 CUDA 计时。不要把稳定报告对象变成调试日志。

第五，pinned buffer 是资源优化，不是无限内存许可证。 `PinnedHostTaskBuffer` 让 `*.mat` 任务更适合派发，但页锁定内存会影响操作系统调页自由度。真正的上界来自 pipeline 的 in-flight 窗口和完成队列背压，而不是 pinned allocation 本身。如果窗口无限增大，高吞吐路径也会变成资源耗尽路径。

第六，边界不应被性能优化轻易打穿。 高性能项目常有一种诱惑：为了少拷贝、少封装、少函数调用，把文件映射地址、pinned buffer、 device pointer、CUDA stream 和输出队列揉成一个大对象。除非有明确测量证据，并且仍能保持语义边界，否则这种优化更像债务而不是性能。好优化应该让特殊情况变成局部策略，而不是让特殊情况污染整个系统。

4.8 小结

本章把前一章的 CUDA kernel 从“实现细节”提升为“系统构件”来观察。它应当像纯函数一样被调用：

$$(A, C) \mapsto (U, \Sigma, V, s),$$

即使其内部不可避免地包含设备内存、kernel launch、布局转置、runtime 同步和错误检查。这个函数式外观不是美学装饰，而是可测试性、可替换性、错误隔离和长期演化的基础。

轻量 DDD 在这里的价值，不是给项目贴企业架构标签，而是提醒我们：真正的领域核心是 SVD 的数值不变量和 CUDA 执行约束；CLI、I/O、pipeline 与 CMake 都是围绕它建立的

边界。 kernel 不认识文件，不认识路径，不认识输出顺序，不认识用户脚本，也不认识构建目录。它只认识矩阵、配置和数值结果。

这也让后续章节自然衔接：下一章讨论 I/O 如何把外部字节转化为矩阵流与 pinned dispatch task；再下一章讨论 pipeline 如何用全局线程池、future 窗口和重排序缓冲区组合输入流、纯函数式 kernel 与输出流；随后 CLI 与 CMake 分别定义用户意图边界和构建 artifact 边界。一个良构 CUDA 数值系统的脊梁就在这里：把最复杂、最昂贵、最脆弱的 kernel 放在最窄、最稳定、最可验证的位置。

5 将内存映射到文件进行流式处理

上一章讨论了 CUDA kernel 如何作为纯函数式构件嵌入良构系统：kernel 只处理矩阵、配置和数值结果，而不应认识文件、路径或用户接口。本章沿着这条边界继续向外走，讨论矩阵如何进入和离开系统。对于端到端 SVD 程序，矩阵如何被验证、如何被转换为主机侧连续内存、如何被输出到下游工具，同样会影响可扩展性（scalability）、可复现性（reproducibility）与故障语义（failure semantics）。

本章聚焦 `io.cpp`。它不是算法层的附属脚本，而是整个系统的数据平面（data plane）：它定义了矩阵流的外部格式、跨平台字节序、文件映射生命周期、读写游标、文本兼容格式以及 policy-based stream 抽象；最新实现还增加了面向计算派发的单游标读取器（single-cursor dispatch reader）与页锁定主机任务缓冲区（pinned host task buffer）。若说 `kernels.cu` 处理“如何计算”，那么 `io.cpp` 处理“什么数据以何种契约进入计算”。在 systems 视角下，这两者同等重要；因为任何计算内核最终都只能处理已经被正确表示、正确边界化、正确拥有的数据。

5.1 设计目标：把 I/O 从批量文件操作重构为矩阵流

`io.cpp` 的首要目标不是提供一个简单的 `read()` / `write()` 包装，而是把文件视为矩阵序列（matrix stream）。外部文件中可以顺序存放多张矩阵，应用层逐张读取、逐张计算、逐张写回。这种接口具有三个系统层面的意义。

第一，降低应用层驻留压力。 若接口强迫调用者一次性读取完整文件，应用层就必须同时持有所有测试矩阵、所有中间结果或所有输出。对于 SVD 这类输出规模接近输入规模的算法，批量驻留很快会放大为

$$O\left(\sum_i m_i n_i\right)$$

级别的长期内存占用。流式接口把边界收缩到“当前矩阵”：

$$\texttt{read_one} \to \texttt{compute} \to \texttt{write_one}.$$

这为下一章的生产者-消费者（producer-consumer）pipeline 提供了自然接口。

第二，稳定文件格式契约。 二进制 `*.mat` 格式采用“头部 + 载荷”的重复结构：

$$(\texttt{rows}, \texttt{columns}, \texttt{payload})^*.$$

其中头部由两个 `uint64_t` 组成，载荷为 $rows \cdot columns$ 个 `double`。每个矩阵都是自描述记录（self-describing record）：读取端不依赖外部索引表，也不需要先扫描全文件才能定位下一张矩阵。这使 EOF、截断、维度溢出和载荷不足都可以在局部被发现。

第三，分离“流控制”与“序列化策略”。 同一个 `MatrixInputStream<Policy>` 可以服务于二进制 `MatFilePolicy`，也可以服务于文本 `TxtFilePolicy`。模板层只关心打开、读取下一张矩阵、写入下一张矩阵和刷新；具体格式如何编码字节、如何解析数字、是否使用内存映射，都被下沉到 policy。这是 policy-based design 的价值：把稳定控制流放在模板里，把易变格式细节放在策略里。

5.2 数据模型与文件格式

I/O 层的核心领域对象是 `Matrix`：

$$\texttt{Matrix} = (rows, columns, values).$$

`values` 使用行主序（row-major order）的一维 `std::vector<double>`。这与主机侧 C++ 容器、文本输出顺序以及 GPU 入口前的数据准备保持一致。矩阵元素索引为

$$index(i, j) = i \cdot columns + j.$$

二进制 `*.mat` 文件的单条记录可抽象为：

$$R = \underbrace{\texttt{uint64_be}(rows) \parallel \texttt{uint64_be}(columns)}_{\text{metadata}} \parallel \underbrace{\texttt{float64_be}(a_{00}) \parallel \cdots \parallel \texttt{float64_be}(a_{m-1,n-1})}_{\text{payload}}.$$

其中 `_be` 表示大端序（big-endian），也就是网络字节序（network byte order）。文件整体为

$$F = R_0 \| R_1 \| \cdots \| R_{k-1}.$$

这一格式选择体现了一个保守但重要的系统设计原则：磁盘格式应当独立于主机 ABI。如果直接把主机内存中的 `double` 与 `size_t` 原样写出，文件就会隐式依赖平台端序、 `size_t` 宽度、结构体填充与浮点表示。当前实现至少显式固定了两个维度：

- 维度字段固定为 64 位无符号整数，而不是平台相关的 `size_t`；
- 所有 64 位模式在磁盘上使用网络字节序，读取时再转换回主机字节序。

代码中 `to_network_u64` 与 `from_network_u64` 通过 `std::endian` 判断本机端序，并在小端机器上执行字节交换（byte swap）。`double` 则先通过 `std::bit_cast<std::uint64_t>` 转换

为 64 位模式，再复用同一套端序逻辑。这比手写按字节拼接更直接，也避免了违反严格别名规则（strict aliasing rule）。

需要诚实指出的是：当前格式隐含假设 `double` 采用 64 位 IEEE 754 binary64 表示。这是现代 x86-64、AArch64 与 CUDA 主机环境中的现实假设，但若要把 `*.mat` 提升为长期归档格式，文件头中还应加入 magic number、format version、element type 与校验信息。本项目当前阶段选择最小头部，是为了降低实现复杂度并保持测试数据可直接检查。

5.3 内存映射：把文件访问交给虚拟内存系统

`MatFilePolicy` 的二进制路径使用内存映射文件（memory-mapped file）。平台相关的系统调用被限制在匿名命名空间内部：

职责	POSIX	Windows
打开文件	`open`, `fstat`	`CreateFileW`
建立映射	`mmap`	`CreateFileMappingW`, `MapViewOfFile`
释放与刷新	`munmap`, `msync`	`UnmapViewOfFile`, `FlushViewOfFile`
调整大小	`ftruncate`	`SetEndOfFile`

内存映射的系统语义可以概括为：

$$\text{file bytes} \longleftrightarrow \text{kernel page cache} \longleftrightarrow \text{process virtual address space.}$$

程序不再围绕 `read()` 系统调用显式搬运固定大小缓冲区，而是获得一段虚拟地址范围。实际页面在访问时由页错误（page fault）按需载入，并由操作系统的页缓存（page cache）管理复用与回写。这通常减少了用户态缓冲区管理、系统调用频率与重复拷贝，但它并不等于算法层面的“零拷贝”。当前实现仍然需要把磁盘上的网络序字节解码为主机侧 `std::vector<double>`，因为后续 CUDA 计算需要连续、可拥有、可传输到设备的数值数组。

5.4 只读映射：不可变字节视图与游标推进

`MemoryMappedInputFile` 是二进制输入路径的资源拥有者。构造时，它完成三件事：

1. 打开文件并查询文件长度；
2. 若文件非空，则建立只读映射；
3. 暴露 `std::span<const std::byte>` 作为不可变字节视图。

析构时，它释放映射视图与底层文件句柄。拷贝构造和拷贝赋值被删除，移动构造和移动赋值被允许。这与前一章中 `DeviceMatrix` 的思想一致：系统资源必须有唯一所有者（unique owner），而不是被多个 C++ 对象模糊共享。这里的资源包括文件句柄、映射句柄和虚拟地址区间；任何一个被重复关闭，都可能变成典型的 use-after-close 或 double-close 错误。

`MatReader::Impl` 在此基础上保存两个状态：

$$(\texttt{bytes}, \texttt{cursor}).$$

cursor 是当前读取位置。每次 MatFilePolicy::read_next 执行时，状态转移为：

$$cursor \xrightarrow{\text{read header}} cursor + H \xrightarrow{\text{read payload}} cursor + H + 8mn,$$

其中 $H =$ sizeof(MatMetaData)。

读取路径的正确性由一组局部检查保护：

- 若 cursor == bytes.size()，返回 EOF；
- 若剩余字节少于头部大小，报告截断头部（truncated header）；
- 将 rows 与 columns 从网络序转换为主机序；
- 检查维度是否超过本机 size_t；
- 使用 checked arithmetic 计算元素数与载荷字节数；
- 若剩余字节少于载荷大小，报告截断载荷（truncated payload）。

这些检查看似朴素，但它们决定了文件格式能否被安全地暴露给系统其余部分。I/O 层不能把畸形输入传递给 GPU kernel 后再期待 kernel “自然失败”。在系统边界处尽早拒绝坏数据，是降低故障半径（blast radius）的基本做法。

5.5 追加式输出映射：用容量管理替代频繁写系统调用

输出路径由 AppendMappedOutputFile 管理。它把文件看作一个可增长的追加缓冲区：

$$(\texttt{data}, \texttt{size}, \texttt{capacity}).$$

其中 size 是已经写入的逻辑字节数；capacity 是当前映射的物理文件容量。每次追加新载荷时，代码先计算所需容量：

$$required = size + |payload|,$$

若 required <= capacity，则直接 memcpy 到映射区尾部；否则按倍增策略扩容：

$$capacity' = \max(2^k \cdot capacity_0, required),$$

其中初始容量下界为 1 MiB。

扩容过程是一个明确的四阶段协议：

1. 刷新当前已写前缀；
2. 取消当前映射；
3. 调整底层文件大小到新容量；
4. 重新建立写映射，并更新 capacity。

析构时，close() 会尝试刷新已写前缀，释放映射，然后把文件截断到逻辑 size。这一步很关键：倍增扩容会让底层文件暂时大于真实内容，最终必须恢复为精确长度，否则下游读取器会把尾部空洞解释为额外记录或截断记录。

从摊还分析（amortized analysis）看，倍增扩容使 N 字节输出的映射重建次数为

$$O(\log N),$$

而普通追加写入路径为 $O(1)$。代价并没有消失：重映射会触发同步、取消映射和文件截断操作，包括 msync / FlushViewOfFile 以及 munmap / UnmapViewOfFile。但相比每个小块都执行系统调用，它把高频路径压缩为用户态内存拷贝，把低频路径交给容量管理。

5.6 编码与解码：分块处理而不是整块临时复制

二进制记录的载荷是 $rows \cdot columns$ 个 64 位浮点模式。由于磁盘使用网络序，读写都需要执行逐元素转换。当前实现采用固定大小分块：

$$B = 32 \cdot 1024 \text{ elements}.$$

读取时，MatFilePolicy::read_next 直接从映射字节区按块取出 64 位模式：

$$encoded_i \to \texttt{from_network_u64} \to \texttt{bit_cast<double>}.$$

每个元素通过 std::memcpy 取出，而不是把 std::byte* 强转为 std::uint64_t*。这避免了未对齐访问（unaligned access）与别名规则风险。

写入时，MatFilePolicy::write_next 先写网络序头部，再复用一个 uint64_t 编码块向量。每个块完成：

$$double_i \to \texttt{bit_cast<uint64_t>} \to \texttt{to_network_u64} \to \texttt{append(bytes)}.$$

这里的分块有两个作用。第一，它限制临时编码缓冲区大小，不随矩阵规模线性增长。第二，它让输出路径可以连续追加多个字节块，而不需要为每张矩阵构造一个完整的编码副本。因此，虽然 Matrix 本身仍然以完整矩阵为粒度进入计算，序列化层的额外空间为

$$O(B),$$

而不是 $O(mn)$。

5.7 派发读取：把字节流先送入页锁定任务缓冲

上面的 MatReader 路径服务于传统矩阵流接口：

$$\texttt{*.mat bytes} \longrightarrow \texttt{Matrix}.$$

新加入的 MatDispatchReader 则面向下一层 pipeline 的任务派发（task dispatch）场景。它不立即把载荷解码为 std::vector<double>，而是把单条 *.mat 记录拆成更接近调度单元的数据结构：

$$\texttt{MatDispatchTask} = (\texttt{sequence_index}, \texttt{rows}, \texttt{columns}, \texttt{buffer}).$$

其中 `buffer` 是 `PinnedHostTaskBuffer`，底层通过 `cudaMallocHost` / `cudaFreeHost` 管理一整块页锁定主机内存（page-locked host memory）。这条路径的核心思想不是替代 `MatInputStream`，而是给更高吞吐的生产者-消费者 pipeline 预留一个更合适的输入形态：主线程可以用一个单调文件游标顺序读取记录，把原始网络序 payload 放进可复用的 pinned buffer，再把解析、计算或异步拷贝交给后续阶段。

数据结构上，特殊情况被折叠成普通区间。 `PinnedHostTaskBuffer` 没有为输入区、解码区、GPU staging 区分别维护多块 allocation，而是把它们表达为同一块连续 pinned memory 内的两个 span：

$$\underbrace{[0, \texttt{input_bytes})}_{\text{raw payload}} \quad \| \quad \underbrace{[\texttt{input_bytes}, \texttt{input_bytes} + \texttt{workspace_bytes})}_{\text{workspace}}.$$

调用者通过 `mutable_input_bytes()` 与 `mutable_workspace_bytes()` 获得两个视图。这是一种很朴素的好品味：不是让每个阶段各自发明一套临时缓冲，而是在 I/O 边界把“输入字节”和“后续工作区” 统一成一个明确拥有者。移动构造和移动赋值只移动指针与大小，拷贝被删除；因此任务可以在线程队列中移动，但不会意外复制 pinned memory。

容量策略上，复用比频繁分配更重要。 `reserve(input_bytes, workspace_bytes)` 先计算总需求：

$$required = input_bytes + workspace_bytes.$$

若已有容量足够，只更新输入区和工作区大小；若容量不足，则按倍增策略增长，初始下界为 64 KiB。因此同一个 `MatDispatchTask` 可以被读取循环反复复用：小矩阵不会导致频繁 `cudaMallocHost` / `cudaFreeHost`，大矩阵也只在超过历史峰值时触发重新分配。这与输出映射的摊还思想一致，只是资源从“文件映射容量”变成了“页锁定主机容量”。

语义上，派发任务保存的是原始载荷而不是已解码矩阵。 `MatDispatchReader::read_next` 的状态转移为：

$$\begin{aligned}\text{read header} &\longrightarrow \text{validate dimensions} \longrightarrow \text{reserve pinned buffer}\\ &\longrightarrow \text{read payload bytes} \longrightarrow \text{assign sequence index.}\end{aligned}$$

载荷仍保持磁盘上的网络字节序。若后续阶段需要传统 `Matrix`，可以调用 `decode_dispatch_task_matrix` 完成逐元素解码；若未来 kernel wrapper 支持直接从 pinned staging buffer 构造输入，则也可以在工作区中完成解码并与 host-device copy 更紧密地衔接。这给 pipeline 留出了演化空间，而没有把 CUDA stream、device pointer 或 kernel 调度塞进 I/O 层。

这里必须保持一个清醒的边界：pinned memory 不是免费资源。页锁定内存不能被操作系统随意换出，过量使用会挤压系统分页能力，甚至拖慢整机。于是 `PinnedHostTaskBuffer` 只能解决“当前任务如何更适合被派发”的问题，不能替代有界队列（bounded queue）和背压（backpressure）。真正的内存上界仍然要由 pipeline 控制：

$$M_{\text{pinned}} \le \sum_{t \in \text{in flight}} \texttt{capacity_bytes}(t).$$

如果 in-flight 任务数无界，pinned buffer 再漂亮也会把系统拖入资源耗尽。因此这条新路径和下一章的生产者-消费者队列是一组配套设计：I/O 层让单个任务适合高速派发，pipeline 层决定同时允许多少任务存在。

5.8 Policy-based stream：把格式差异从控制流中剥离

`io.hpp` 中定义了两个 concept：

$$\texttt{MatrixInputPolicy} = \{\texttt{Reader}, \texttt{open_reader}, \texttt{read_next}\},$$
$$\texttt{MatrixOutputPolicy} = \{\texttt{Writer}, \texttt{open_writer}, \texttt{write_next}, \texttt{flush}\}.$$

它们把“什么是合法矩阵输入策略/输出策略”表达为编译期契约（compile-time contract）。模板类 `MatrixInputStream<Policy>` 与 `MatrixOutputStream<Policy>` 只依赖这一契约，并不依赖具体文件格式。

这一层的状态机非常小：

$$\texttt{MatrixInputStream} = (\texttt{reader}, \texttt{eof}),$$

$$\texttt{MatrixOutputStream} = (\texttt{writer}).$$

输入端的 `read_one` 调用 policy 的 `read_next`；若返回 `false`，流进入 EOF 状态，后续读取直接返回 `false`。输出端的 `write_one` 调用 `write_next`，`write_all` 则在循环后统一 `flush`。

这种结构的关键收益不是“模板很漂亮”，而是依赖方向正确：

> 流控制依赖抽象 policy；格式实现依赖底层文件机制；应用层依赖流接口。反过来的依赖都不存在。

因此，新增一种格式，例如压缩二进制格式或 NumPy `.npy` 子集，并不需要修改 pipeline 的控制流；只需要提供满足 concept 的新 policy。这是 C++20 concept 在系统设计中的实际价值：它把运行期协议错误提前为编译期接口错误。

二进制 `*.mat` policy：吞吐优先的主路径。 `MatFilePolicy` 是这套抽象的“正常路径”。它把前面已经讨论过的内存映射、固定头部、网络字节序编码和追加式输出统一封装到 `open_reader` / `read_next` / `open_writer` / `write_next` / `flush` 这组接口后面。于是对上层而言，`*.mat` 只是“一个满足 concept 的 policy”，而不是一串散落在控制流里的特殊分支。

这件事的系统意义在于：高性能实现被局部化了。页缓存、映射重建、字节序转换、分块编码这些复杂性都留在 `MatFilePolicy` 内部；`MatrixInputStream` / `MatrixOutputStream` 仍然只维持一个极小状态机。换句话说，性能优化存在，但它不污染抽象边界。

文本 `*.txt` policy：可检查性优先的兼容路径。 `TxtFilePolicy` 提供文本矩阵流。其格式约定是：行内元素用空格分隔，行之间用换行分隔，矩阵之间用空行分隔。读取器逐行解析：

- 连续空白行在矩阵开始前被忽略；
- 一旦进入矩阵，空白行表示当前矩阵结束；

- 非空行必须至少解析出一个浮点数；
- 同一矩阵内每行列数必须一致；
- 文件结束时，若当前正在读取矩阵，则返回最后一张矩阵。

文本路径使用 `std::locale::classic()`，避免本地化小数点或分隔符影响解析。写入时使用 `std::numeric_limits<double>::max_digits10` 精度，以尽量保证二进制浮点值经十进制文本输出再读入后可以 round-trip。

从性能角度看，文本路径显然不是高吞吐路径。它需要执行十进制解析、格式化和更多分支判断。但它在系统中仍然有价值：测试样例可读、调试输出可检查、用户可以手写小矩阵输入。因此，把它放进 policy 层的意义，不是把文本伪装成“和二进制一样快”，而是让 `*.txt` 也通过同一套流接口接入系统。这样，`*.mat` 面向吞吐与稳定格式， `*.txt` 面向可观察性（observability）与开发便利；二者共享抽象，不共享性能假设。

5.9 错误模型：在边界处失败，而不是在深处腐化

`io.cpp` 的错误模型以异常（exception）报告不可恢复错误。这里的异常不是控制流装饰，而是对系统边界失败的直接编码。典型错误包括：

错误类别	检测位置	系统含义
文件打开失败	构造 reader / writer	外部资源不可用
文件长度非法	映射前	底层文件状态不满足前置条件
头部截断	`read_next`	记录边界被破坏
载荷截断	`read_next`	元数据与实际字节数不一致
维度溢出	checked arithmetic	输入会导致本机地址空间或容器越界
pinned 分配失败	`PinnedHostTaskBuffer::reserve`	主机侧 staging 资源不足
派发载荷截断	`MatDispatchReader::read_next`	任务边界被破坏，不能进入计算队列
行列不一致	文本解析	文本矩阵不是矩形结构
刷新失败	`flush`	输出持久化状态未知

特别值得注意的是 checked arithmetic。矩阵载荷大小由

$$bytes = rows \cdot columns \cdot 8$$

决定。若这个乘法在 `size_t` 中溢出，后续检查会得到一个错误的小值，进而可能接受畸形文件。 `checked_multiply` 与 `checked_add` 把这种风险变成显式异常。这是一种很“朴素”的代码，但在系统边界上非常有品味：不要让整数溢出把坏输入伪装成好输入。

5.10 性能模型：mmap 优化的是数据通路，而不是算法复杂度

设某张矩阵有 m 行、n 列。二进制读取的主要 CPU 工作为：

$$O(mn)$$

次 64 位载荷解码和一次 $O(1)$ 头部解析。写入类似，为 $O(mn)$ 次编码和追加。内存映射不改变这一渐近复杂度；它改变的是常数项、系统调用结构与缓存行为。

与显式 `read()` 循环相比，mmap 路径具有几个潜在优势：

- 顺序扫描映射区时，操作系统可以执行预读（read-ahead）并复用页缓存；
- 应用层不需要管理输入缓冲区生命周期；
- 多个进程读取同一文件时，页缓存可以共享底层物理页；
- 输出追加的高频路径退化为对映射区的 `memcpy`。
- 派发读取路径可以把原始 payload 放入 pinned buffer，为后续异步 host-device copy 提供必要前提。

但同样存在代价：

- 首次访问页面会产生 minor 或 major page fault；
- 大文件映射会消耗虚拟地址空间，尤其在地址空间受限平台上需要谨慎；
- 输出扩容需要解除映射、调整文件大小并重新映射；
- `msync` 或 `FlushViewOfFile` 的同步语义可能成为尾部延迟来源；
- 网络序转换仍然是 CPU 上的逐元素循环，不能被 mmap 自动消除。
- pinned memory 会降低操作系统调页自由度，必须通过任务复用和有界 in-flight 数量控制总量。

因此，本章的结论不应是“mmap 总是更快”。更准确的说法是：当前 I/O 层用 mmap 把文件访问纳入虚拟内存管理，使顺序二进制矩阵流获得较低的用户态管理复杂度；实际吞吐仍取决于存储设备、页缓存命中率、矩阵规模、编码转换成本和 pipeline 是否能与 GPU 计算重叠。

5.11 与计算 pipeline 的接口边界

从端到端视角看，I/O 层向上提供的是一组非常小的语义：

```
bool read_one(Matrix&)

void write_one(const Matrix&)

void flush()
```

这个接口刻意没有暴露 mmap、文件句柄、字节序或文本解析细节。应用层只看到矩阵对象与 EOF。这种边界使 pipeline 可以关注自己的核心问题：如何把输入矩阵送入 CUDA SVD，如何把输出矩阵交给消费者线程，如何在计算与写入之间建立背压（backpressure）。

新增的派发接口则更接近高性能 pipeline 的内部边界：

```
bool read_next(MatDispatchTask&, workspace_bytes)

Matrix decode_dispatch_task_matrix(const MatDispatchTask&).
```

它比 `read_one(Matrix&)` 暴露更多信息：记录序号、原始 payload、预留工作区与 pinned allocation。这不是抽象泄漏，而是层间契约的有意细化。传统流接口适合“读出一张矩阵再计算”的简单路径；派发接口适合“先把一条记录变成可移动任务，再由后续阶段决定何时解码、何时拷贝、何时计算”的流水线。两者共同存在，使项目不必为了高性能路径破坏已有的兼容 API。

与此同时，I/O 层也没有越界承担 GPU 责任。它不会分配 device memory，不会启动 kernel，不会管理 CUDA stream，也不会为了“优化”而把文件映射地址直接交给设备侧。这样的克制很重要：文件格式、主机内存、设备内存、并发 pipeline 是不同抽象层。若把它们混在一个对象里，短期看似减少了函数调用，长期会制造难以测试的生命周期耦合。

5.12 当前实现的边界与可演化方向

当前 `io.cpp` 已经建立了清晰的数据平面，但它仍有几个需要明确记录的边界。

第一，流式粒度是矩阵级，而不是元素级。 `MatReader` 映射整个文件，`read_next` 每次解码一整张矩阵到 `std::vector<double>`。这对 dense SVD 是合理的，因为 GPU kernel 本身需要完整矩阵参与迭代。但若未来处理的是超大矩阵的分块算法，I/O 层就需要提供 tile stream 或 block stream，而不是 matrix stream。

第二，文件头还不是长期稳定格式。 当前 `*.mat` 没有 magic number、版本号、校验和、元素类型枚举或压缩标志。这使格式极简，但也意味着错误文件可能只能通过截断或维度异常被发现。若该格式要跨项目复用，建议将记录头扩展为：

```
magic, version, dtype, rows, columns, payload_bytes.
```

第三，输出映射扩容是同步路径。 倍增策略把重映射次数降到对数级，但每次重映射仍然会刷新、取消映射、截断文件并重新映射。若输出吞吐成为瓶颈，可以考虑两类优化：一是基于输入规模预估输出总大小并一次性预分配；二是在 producer-consumer pipeline 中让写线程批量聚合结果，降低 flush 频率。

第四，文本路径强调可读性，不强调吞吐。 `TxtFilePolicy` 对开发体验很有价值，但它不应被误认为高性能输入路径。大规模实验应优先使用 `*.mat`；文本格式更适合小样例、单元测试和人工检查。

第五，派发读取路径目前只覆盖 `*.mat`。 `MatDispatchReader` 是二进制格式的专用优化路径，因为它依赖固定大小头部与可直接计数的二进制 payload。文本格式需要逐行解析和列数校验，不适合原样塞入同一种 dispatch task。若未来要对文本输入也做异步读取，更合理的做法是先

把文本解析成 `Matrix` 或独立的 parsed task，而不是扭曲 `MatDispatchTask` 去同时表达两种完全不同的载荷语义。

第六，pinned buffer 只是 host-device 重叠的前置条件。 页锁定内存让异步拷贝更有意义，但它本身不会自动带来 CUDA stream 重叠。要真正实现

$$\text{read/decode} \parallel \text{host-device copy} \parallel \text{kernel} \parallel \text{write}$$

还需要 kernel 层提供异步接口，并由 pipeline 明确管理 stream、事件与任务生命周期。因此当前 `PinnedHostTaskBuffer` 应被理解为一块稳固地基，而不是完整的异步执行框架。

第七，端序被固定，浮点语义仍依赖现实平台。 网络字节序解决了字节排列问题，但没有把 `double` 的语义完全自描述化。在现代 HPC 环境中这通常足够；但若追求跨几十年的归档稳定性，文件格式应显式声明 IEEE 754 binary64。

5.13 小结

本章把 `io.cpp` 定位为系统的数据平面（data plane）：它不实现 SVD，不编排 pipeline，也不管理 CUDA 资源，而是把外部持久化介质抽象为一个类型安全的矩阵流。围绕这一目标，它同时承担四类边界职责：定义二进制 `*.mat` 与文本格式契约，使用内存映射文件（Memory-Mapped Files）降低顺序访问的用户态管理成本，执行网络字节序与 `double` 载荷之间的编码转换，并在文件损坏、维度非法、映射失败或写入失败时尽早停止。

从系统设计角度看，I/O 层的价值不在于“读写文件”这件小事本身，而在于它把脏世界关在边界外：坏输入不能越过 I/O 边界，资源不能泄露或重复释放，格式变化不能污染 pipeline 控制流，高吞吐路径也不能牺牲可验证性。这样，下一章的 pipeline 才能把输入流、CUDA SVD kernel 与输出流组合起来，而不必理解字节序、mmap 生命周期或文本解析细节。

这正是数值系统中经常被低估的部分。GPU kernel 可以把浮点吞吐推到很高，但系统最终的可信度，往往取决于这些更安静的边界代码：它们决定数据是否被正确解释，错误是否被及时停止，以及未来的扩展是否还能保持清楚的依赖方向。

6 全局线程池、重排序缓冲区与 Pipeline 聚合根

前几章已经把数值内核（kernel）与 I/O 数据平面（data plane）拆开了；本章只讨论 `pipeline.cpp`。它不新增任何奇异值分解（Singular Value Decomposition, SVD）公式，也不重新发明文件格式，而是负责把

$$\texttt{testcases} \longrightarrow \texttt{kernel} \longrightarrow \texttt{output}$$

拼成一次端到端执行。换句话说，`JacobiSvdPipeline` 是应用层聚合根（aggregate root）：它拥有调度权、关闭协议与错误传播语义，但不偷走 I/O 与 CUDA 内核的领域职责。

本章所讨论的并发骨架，采用一种以 `future` FIFO 队列实现的重排序缓冲区（reorder buffer, ROB）。它把“乱序完成、顺序提交”这层语义折叠进一个 **按输入顺序排队的 `future`**

队列里。于是整体骨架可以写成

$$\texttt{input} \longrightarrow \texttt{dispatch} \longrightarrow \texttt{global thread pool} \longrightarrow \texttt{FIFO future queue} \longrightarrow \texttt{single writer}.$$

如果借用处理器体系结构（computer architecture）的语言，这对应一个执行顺序与提交顺序分离、并以有序提交（ordered commit）收敛结果的 ROB。输入顺序必须保真，工作线程可以乱序完成，而文件输出必须回到确定性的提交顺序。本章就围绕这层语义，展开 pipeline 的理论动机、控制流与工程边界。

6.1 为什么这样设计 pipeline：理论动机、顺序契约与复杂度边界

先看最朴素的串行方案。若第 i 个 testcase 的主要主机侧耗时分解为

$$R_i = \text{读取/派发}, \qquad D_i = \text{解码}, \qquad C_i = \text{SVD 计算}, \qquad W_i = \text{写出},$$

那么串行总耗时近似为

$$T_{\text{serial}} = \sum_{i=0}^{N-1} (R_i + D_i + C_i + W_i).$$

这种写法最容易理解，但它把文件读取、页锁定缓冲准备、GPU 计算与结果落盘全部压在一条长临界路径（critical path）上。只要某一张矩阵稍慢，后面所有 testcase 都会被迫等待。

当前 pipeline 的思路不是在主机侧“暴力多线程化一切”，而是只把真正可并行的 *跨 testcase 工作*交给全局线程池（global thread pool），并保留一个单线程 writer 负责稳定输出。于是稳定阶段的端到端节拍更接近

$$T_{\text{steady}} \approx \max(T_{\text{dispatch}}, T_{\text{workers}}, T_{\text{commit}}),$$

其中最后一项的名字特意写成 `commit` 而不是 `write`，因为写线程当前做的不只是“把字节写出去”，它还承担了一个更强的顺序承诺： **只有排在最前面的 future 可以提交，后面的结果即使先算完，也不能越过它。**

这就引出了顺序契约（ordering contract）。输入流天然有序：

$$X_0, X_1, \ldots, X_{N-1}.$$

工作线程一旦并发执行，真实完成顺序就会变成某个排列

$$P_{\pi(0)}, P_{\pi(1)}, \ldots, P_{\pi(N-1)}.$$

系统采用如下顺序契约： **主线程按照输入顺序，把 `future<OutputPacket>` 依次送进输出阶段；写线程也按这一顺序依次取 future，并在提交点调用 `future.get()`。** 因此真正被提交到文件中的顺序不是“谁先算完”，而是

$$\texttt{commit}(0) \prec \texttt{commit}(1) \prec \cdots \prec \texttt{commit}(N-1).$$

这一定义下，ROB 不通过显式散列表维护“谁已经完成但不能提交”的状态，而是用队列顺序天然表达“谁有资格先提交”。

当然，这种 ROB 设计也不是免费午餐。它把原本的“显式按 index 回填复杂度”换成了“队首阻塞（head-of-line blocking）”。若 future_0 迟迟不完成，而 future_1、future_2 已经 ready，写线程仍然必须在

$$\texttt{future}_0\texttt{.get()}$$

处等待。这意味着当前实现优化的是**状态简洁性与顺序确定性**，而不是“谁先完成谁先提交”的最大化吞吐。对本项目来说，这是一个很合理的工程取舍：实验结果的稳定顺序比额外引入一套乱序完成表更重要。

第三个约束是内存与 in-flight 上界。CLI 把 `--queue-capacity` 解释为

$$W = \max(\texttt{max_queued_results}, 1).$$

但必须精确说明：这里的 W 直接限制的是 `FutureQueueOutputStage` 内部 `future_queue_` 的容量，而不是“所有尚未写出的对象总数”的严格数学上界。这是因为 writer 线程会先把一个 future 从队列里弹出，再在队列外执行 `get()`；主线程也可能在 `submit()` 被阻塞前，刚刚多提交了一个对应 worker 任务。于是更准确的说法是：

$$M_{\text{pending}} = O((W + \delta) \cdot E_{\max}),$$

其中 $E_{\max}$ 表示单个 testcase 结果包的最大规模，δ 是由“writer 手中正在等待的一个 future”和“生产者临界区外的一次过渡提交”引入的小常数。这不是 profiling 精确值，而是一条更诚实的系统结论：系统仍然是有界的，但边界来自“有界 future admission”。

最后一个约束是错误诚实性（error honesty）。并发系统最怕的不是抛异常本身，而是某个线程已经知道结果不可信，而主线程还在继续把流程包装成“成功执行完毕”。本章后面的所有细节，无论是关闭协议、原子计数、还是 writer 线程统一吸收 future 异常，本质上都服务于同一个目标：**一旦任一阶段认定结果不再可信，`run()` 最终就不能报告成功。**

6.2 全局线程池实现：把并发入口集中起来，而不是到处偷开线程

`GlobalThreadPool` 依然刻意保持极小语义集合。它是一个惰性初始化（lazy initialization）的进程内全局对象：

$$\texttt{global_thread_pool()} \Rightarrow \texttt{static GlobalThreadPool pool(default_thread_pool_size())}.$$

默认线程数来自 `std::thread::hardware_concurrency()`，若运行环境不给出可信值，则回退到 4。构造时一次性创建 worker；析构时设置 `stopping_`，唤醒所有线程，再逐个 `join()`。

它的工作线程主循环几乎可以压缩成三步：

$$\text{wait until } \texttt{stopping_} \vee \neg Q.\mathit{empty}() \longrightarrow \text{pop one task} \longrightarrow \text{execute outside lock.}$$

这里最重要的好品味（good taste）不是“用了线程池”这件事本身，而是线程池**不理解业务顺序语义**。它只知道自己维护 `std::queue<std::function<void()>>`，并通过条件变量（condition variable）协调 worker 睡眠与唤醒。至于 testcase 是第几号、输出该不该有序、某个 future 何时能提交，都不属于线程池的职责。

`submit()` 返回的不是 `std::future<void>`，而是模板化的

$$\texttt{std::future<ResultType>}.$$

具体做法是把调用者传入的可调用对象（callable）包装成 `std::packaged_task<ResultType()>`，再把该 task 的执行闭包压入线程池队列。这有两个直接收益。

- 线程池不需要知道结果类型是什么；它只执行闭包，不解释 `OutputPacket`。
- worker 里抛出的异常会自动落入 future 的共享状态（shared state），后续由 writer 线程在 `future.get()` 处统一观察。

这点很关键，因为当前实现已经把“错误汇聚点”从主线程转移到了输出阶段。主线程提交给线程池的闭包通常长成下面这种形式：

$$\texttt{task} \longrightarrow \texttt{decode} \longrightarrow \texttt{kernel.execute} \longrightarrow \texttt{return OutputPacket}.$$

线程池只负责把这个函数跑完；它既不会主动 `get()` 结果，也不会替 pipeline 写文件。

为什么线程池继续保留为全局对象，而不是每次 `run()` 都新建一批线程？核心原因仍然有三点：

- 线程创建与销毁不是零成本；
- 并发总量需要有集中入口，否则每层各自“只多开一点点线程”很快就会失控；
- pipeline 是应用层协议，不应重复建造同一种宿主基础设施。

当然，也要诚实指出它的边界。当前线程池内部任务队列本身并**没有**容量上界，真正的背压主要来自外层的 `FutureQueueOutputStage::submit()`。这意味着：只要所有任务都遵循“提交一个 worker future 后立刻尝试送入输出阶段”的当前控制流，系统的 in-flight 数量就是有限的；但如果未来有别的模块直接复用这同一个全局线程池并大量投递任务，那就需要重新讨论 bounded queue、独立执行器（executor）或更细的资源隔离。

6.3 流程与重排序缓冲区：主线程派发、工作线程执行、写线程恢复顺序

这一节讨论的重排序缓冲区，是“按输入顺序排队的 future 提交队列 + 单 writer 顺序 `get()`”。从抽象效果看，它实现的是 ordered commit；从实现形态看，它是一个 FIFO future-queue ROB。

从控制流看，`run()` 的主骨架现在更接近下面这条链：

$$\begin{aligned}\text{normalize config} &\longrightarrow \text{read testcase } i \longrightarrow \text{submit worker future}\\ &\longrightarrow \text{enqueue future in FIFO output stage} \longrightarrow \text{writer pops next future}\\ &\longrightarrow \texttt{future.get()} \longrightarrow \text{write } U_i, \Sigma_i, V_i.\end{aligned}$$

这里最值得注意的一点是：**输出阶段接收的不是已经完成的 `OutputPacket`，而是未来会产生该 packet 的 `future`**。因此“消费者”其实消费的是提交资格，而不是现成结果。

第一步：主线程保留单游标读取权。 对于 `*.mat` 路径，`io::MatDispatchReader` 顺序读出 `MatDispatchTask`。该对象里已经带有

$$(\texttt{sequence_index}, \texttt{rows}, \texttt{columns}, \texttt{buffer})$$

这组信息，其中 `buffer` 是由 `cudaMallocHost` 管理的页锁定主机内存（page-locked host memory, pinned host memory）。主线程每次把读出的 dispatch task 移动到局部对象 `task_for_worker`，随后立刻把原变量重置为空任务，以保持所有权边界清晰。

对于 `*.txt` 路径，则由 `TextTestcaseSource` 在主线程中直接解析出 `io::Matrix`，再把该矩阵移动给 worker。也就是说，当前实现把 `*.mat` 与 `*.txt` 的差异放在“任务准备形态”这一层，而不是强行让二者共享一套低层数据结构。

第二步：worker 闭包只负责单 testcase 的局部工作。 两条输入路径最终都归约到同一件事：提交一个返回 `OutputPacket` 的闭包。在 `*.mat` 路径里，闭包会先执行

$$\texttt{io::decode_dispatch_task_matrix(task)},$$

把网络字节序载荷解码为 `io::Matrix`；在 `*.txt` 路径里，这一步已经提前在主线程完成。随后两条路径都调用

$$\texttt{kernel.execute(testcase, testcase_index)},$$

其中 `KernelStage::execute()` 先检查

$$rows > 0, \qquad columns > 0, \qquad |\texttt{values}| = rows \cdot columns,$$

再调用 `one_sided_jacobi_svd()`，最后把结果组织成

$$P_i = (i, s_i, U_i, \Sigma_i, V_i).$$

这里的 `testcase_index` 仍然被保存在 `OutputPacket` 里，但 writer **并不依赖该 index 做重排**；它更像一种审计字段（audit field）与调试字段。

第三步：future 在生成后立刻进入输出阶段，而不是先在主线程积压。 `run()` 并没有维护额外的 `inflight deque`。相反，主线程在每次

$$\texttt{pool.submit(...)} \longrightarrow \texttt{future<OutputPacket>}$$

之后，会立即执行

$$\texttt{output.submit(std::move(packet_future))}.$$

因此，主线程不手工轮询一串 future 来决定“哪一个先 ready”，而是把这个顺序资格整体转交给输出阶段。这让主线程的责任进一步收缩成“顺序读取、顺序投递”。

第四步：`FutureQueueOutputStage` 用 FIFO future 队列表达 ROB 的 ordered commit。 输出阶段内部最核心的状态不是“已完成 packet 映射表”，而是

$$Q_f = \texttt{future_queue_}.$$

`submit()` 的协议是：只有当

$$|Q_f| < \texttt{queue_capacity_} \quad \vee \quad \texttt{closed_} \quad \vee \quad \texttt{worker_error_} \neq \varnothing$$

时，生产者线程才能继续。否则主线程会在条件变量上等待。一旦 admission 成功，该 future 就按输入顺序被压入 FIFO 队列。

writer 线程的主循环则执行另一条极简状态机：

$$\text{pop front future} \longrightarrow \texttt{future.get()} \longrightarrow \texttt{write_packet(packet)} \longrightarrow \texttt{written_count_++}.$$

`ResultWriter::write_packet()` 又进一步固定了每个 testcase 的落盘顺序：

$$U_i \longrightarrow \Sigma_i \longrightarrow V_i.$$

所以，这个 ROB 的“顺序恢复”不是通过比较 `packet.testcase_index` 完成的，而是通过**谁先被压入 future 队列，谁就先拥有提交资格**来完成的。

第五步：队首阻塞是显式接受的系统属性。 由于 writer 会在拿到队首 future 后直接调用 `get()`，所以它天然具备以下性质：

$$\text{if } \texttt{future}_i \text{ not ready, then commit}_{i+1} \text{ cannot happen.}$$

换句话说，后面的 testcase 即使已经算完，也不能越过更早的 testcase 提交。这说明系统采用的是有序提交窗口，而不是“谁先完成谁先提交”的策略。

这套机制的好处是状态少、推理简单、错误路径更短。坏处也同样真实：一旦某个早期 testcase 特别慢， writer 就会在它前面卡住，后续 ready 的结果只能在 future 共享状态中等待。所以这里不存在所谓“纯优势升级”；它是一个非常典型的 systems trade-off： **用更小的协议状态，换取可接受的 head-of-line blocking。**

第六步：背压沿着生产者-消费者链条自动回传。 一旦 writer 比较慢，或者某个队首 future 长时间不 ready， `future_queue_` 就会逐步占满。此时主线程会在 `output.submit()` 处阻塞，从而形成如下背压链：

$$\text{slow commit or slow writer} \rightarrow Q_f \text{ full} \rightarrow \text{producer blocked in submit} \rightarrow \text{dispatch slows down.}$$

这条回路很重要，因为它意味着慢写出不会被伪装成“只是多用了一点内存”。当前实现明确选择让吞吐退化显式发生，而不是让资源占用隐式膨胀。

6.4 工程健壮性细节：配置归一化、关闭协议、异常传播与不变量

并发骨架搭好之后，真正决定工程可用性的，是这些“看起来不起眼、但实际上最容易埋雷”的细节。当前 `pipeline.cpp` 的健壮性，主要建立在四组机制上。

其一，先归一化配置，再启动并发。 `run()` 一开始就检查输入/输出路径是否为空，再通过 `resolve_file_format()` 把 `auto_detect` 解析成 `mat` 或 `txt`。若用户启用了布局转置自动调优（layout-transpose auto-tuning），并且模式仍是 `LayoutTransposeMode::auto_select`，pipeline 会先调用 `auto_tune_layout_transpose_threshold()`，把推荐阈值写回本次运行使用的 `runtime_kernel_config`，然后把 `layout_transpose_auto_tune` 关掉。

这一步必须发生在任何 worker 启动之前。否则多个线程若各自拿着“尚未定型”的配置去 benchmark，那同一次运行内部就会出现语义竞争（semantic race）：大家计算的并不是同一个程序。

其二，成功关闭与失败关闭是两条不同协议。 成功路径上，主线程读完整个输入流后调用

$$\texttt{output.close_success(submitted_count)}.$$

这会设置

$$\texttt{expected_count_} = \mathit{submitted_count}, \qquad \texttt{closed_} = \mathit{true},$$

唤醒等待中的线程，随后 `join()` writer 线程。 writer 结束后，系统还会额外检查

$$\texttt{written_count_} = \texttt{expected_count_}.$$

若不相等，就直接抛错，而不是把缺包输出误判为成功。

失败路径上，任意主线程异常都会进入 `run()` 的外层 `catch`，随后调用

$$\texttt{output.close_error(std::current_exception())}.$$

这会把首个错误保存到 `worker_error_`，同时设置

$$\texttt{closed_} = \mathit{true}, \qquad \texttt{abort_} = \mathit{true},$$

再唤醒所有等待者并尝试回收 writer 线程。也就是说，**关闭协议不是“一个 bool 完事”，而是明确区分 success close 与 error close。**

其三，异常传播的汇聚点已经转移到输出阶段。 这一点与旧文档最不一样。当前 worker 任务抛出的异常，并不是由主线程直接 `future.get()` 观察到的，而是由 writer 线程在

$$\texttt{future_packet.get()}$$

处统一观察。无论异常来自 dispatch task 解码失败、矩阵布局校验失败、SVD kernel 抛错，还是 worker 闭包内部的任何其他异常，它们都会先进入 future 共享状态，然后在 writer 线程被重新抛出。随后 `consumer_main()` 的总 `catch(...)` 会把该异常保存到 `worker_error_` 中，并设置关闭/中止标志。

writer 自己的 I/O 异常同样走这条路径。例如目录创建失败、输出流构造失败、`write_packet()` 失败、 `flush()` 失败，最后也都会变成

$$\texttt{worker_error_} \neq \varnothing.$$

之后，主线程在后续 `submit()`、`close_success()` 或 `close_error()` 中都会被重抛。于是异常传播图更接近：

$$\begin{aligned}\text{worker failure} &\to \texttt{future shared state} \to \texttt{writer thread get()}\\ &\to \texttt{worker_error_} \to \texttt{submit/close rethrow} \to \texttt{run fails},\\ &\text{writer failure} \to \texttt{worker_error_} \to \texttt{run fails}.\end{aligned}$$

这比“主线程自己轮询每个 future”更集中，也更符合当前有序提交模型。

其四，RAII 保证资源回收，显式协议保证语义诚实。 `GlobalThreadPool` 的析构负责停止并 `join()` 工作线程；`FutureQueueOutputStage` 的析构则调用 `close_noexcept()` 作为兜底，即使调用者忘记显式关闭，也尽量避免资源泄漏或悬挂线程。但这里必须区分两种责任：

- RAII（Resource Acquisition Is Initialization）负责“不要泄漏资源”。
- `close_success()` / `close_error()` 负责“不要伪造成功语义”。

也正因为如此，析构函数里的兜底关闭选择了 `noexcept` 吃掉异常；真正应该向上报告的错误，必须出现在显式关闭路径中，而不是留给栈展开（stack unwinding）去碰运气。

其五，不变量比技巧更重要。 当前实现真正依赖的不变量（invariants）其实并不多，但每一条都很硬：

- 主线程按输入顺序生成并提交 future；
- 每个成功 worker 恰好返回一个 `OutputPacket`；
- writer 只按 FIFO 顺序消费 future，不做额外排序；
- 每个 testcase 固定写出三张矩阵：$U/\Sigma/V$；
- `total_sweeps` 只用于聚合统计，因此使用 `memory_order_relaxed` 即可；
- 任意线程感知到的不可信状态，最终都必须让 `run()` 失败。

并发代码最危险的地方，往往不是锁多不多，而是状态与例外太多。当前实现能保持可分析性，正是因为它把关键状态压缩成了：一个全局线程池、一个有界 future 队列、一个 writer 线程、一组关闭标志和一个异常指针。状态空间越小，正确性才越容易被真正推理，而不是靠祈祷。

6.5 当前实现的边界与可演化方向

在这套 future-queue 并发模型下，`pipeline.cpp` 的边界也相当清晰。这些边界不是“将来一定要修”的 bug 列表，更像是后续演化时必须显式作答的设计问题。

第一，queue_capacity 限制的是 output-stage admission，而不是线程池内部队列。 当前主线程的确会在每次生成 future 后立刻尝试 `output.submit()`，因此系统不会无限制地向前推进；但线程池内部的 `queue_` 仍然是无界的。换句话说，当前资源控制依赖的是“外层控制流很克制”，而不是“底层执行器自带硬上限”。如果未来还有别的模块复用这同一个全局线程池，现有的资源边界分析就需要重新做。

第二，ROB 的 ordered commit 天然带来 head-of-line blocking。 当前设计最大优点是顺序语义清爽，最大代价也是顺序语义太清爽：一旦较早的 testcase 明显更慢，writer 就必须在它的 future 上等待，后面已经 ready 的 packet 不能越过它。若未来 profiling 表明这类阻塞成为关键瓶颈，那才值得考虑改写成显式完成包缓存、按 index 提交的另一种 ROB 形式。但那会立刻把状态机重新变复杂，因此不应在没有证据时过早优化。

第三，主机侧并发不等于设备侧并发。 多个 worker 线程可以同时调用 `one_sided_jacobi_svd()`，但这并不自动意味着 GPU 上就会出现理想的 kernel 重叠（kernel overlap）。设备端是否真正并行，仍取决于 CUDA runtime、stream 使用方式、显存压力和底层硬件资源。当前 pipeline 优化的是**端到端任务组织**，而不是承诺单 GPU 上一定存在某种特定重叠模式。

第四，`*.txt` 路径仍然把解析留在主线程。 这让文本输入的错误定位和控制流都更简单，也更符合“文本是兼容路径而不是高吞吐路径”的定位。但它同时意味着：若输入瓶颈发生在文本解析上，当前并发模型并不会替你 magically 消除它。未来若真要并行化文本路径，需要单独设计解析任务边界，而不是把当前 `TextTestcaseSource` 机械丢进线程池。

第五，`OutputPacket::testcase_index` 目前更多是元数据，而不是提交控制键。 这里值得记一笔：index 只是“这个 packet 对应谁”的事实记录，而不是 writer 的排序依据。如果将来有人看到该字段就自然假设“writer 靠它排序”，那就会读错代码。

第六，当前报告仍然偏向语义摘要，而不是完整遥测。 `PipelineReport` 会返回 testcase 数、已写矩阵数、总 sweep 数以及布局转置运行时配置；但它还不告诉我们 writer 在多少次 `future.get()` 上发生了明显等待，也不告诉我们 future 队列的峰值占用和队首阻塞持续时间。如果未来要严肃研究 pipeline 吞吐瓶颈，更合理的方向是增加 profiling/telemetry 结构，而不是继续把 `PipelineReport` 膨胀成调试日志容器。

第七，当前失败语义仍然是保守的 fail-fast。 一旦 reader、worker 或 writer 任一环节确认失败，整次 `run()` 都会被判定为失败。系统不会尝试“跳过坏 testcase，继续写后面的好结果”，也不会承诺部分输出文件具有某种恢复协议（recovery protocol）。这种选择有利于实验结果的可审计性，但若未来要支持部分恢复、断点续跑或按 testcase 容错，就必须同步重写输出格式、报告语义与关闭协议。

6.6 小结

本章把 `pipeline.cpp` 重新定位为应用层聚合根（aggregate root）：它既不实现底层 I/O，也不参与单矩阵 SVD 的数值推导，而是负责把顺序输入、并发计算与确定性输出组合成一个可验证的执行协议。这一协议的关键不在于“线程更多了”，而在于**重排序缓冲区被表达为按输入顺序提交的 future 队列**：writer 在固定提交点上执行 `future.get()`，由此实现 ROB 的 ordered commit。

因此，这一章真正的重点并不在于几个类名，而在于新的并发模型如何平衡三件事：较高吞吐、稳定输出顺序和失败语义的诚实传播。它用全局线程池集中 host 侧并发，用有界 future admission 施加背压，用单 writer 保证文件顺序，再用显式关闭协议把所有线程中的异常收拢回 `run()`。从 systems 角度看，这是一种更收缩、也更容易推理的设计；代价则是明确接受队首阻塞。这份取舍本身，就是当前 `pipeline.cpp` 的核心思想。

7 CLI 表示层：从用户意图到 Pipeline 契约

前一章已经把 `pipeline.cpp` 解释为应用层聚合根（aggregate root）：它连接顺序读入、并行计算、重排序缓冲区与单 writer。本章再向外走一层，讨论 `main.cpp` 这个命令行接口（command-line interface, CLI）表示层（presentation layer）。它的任务不是“顺手启动程序”，而是把用户在终端中写下的字符串序列

$$argv = (a_0, a_1, \ldots, a_{n-1})$$

约束成一份可验证、可记录、可交给 pipeline 的执行契约。

与项目早期相比，当前 CLI 已经明显更像一个小型协议层：它负责格式推断、默认输出路径生成、覆盖保护、 `dry-run` 配置展示、布局转置策略与自动调优开关、文本/JSON 双报告，以及参数错误与执行错误的退出码分离。因此，本章不再按“函数枚举”的方式展开，而是集中回答六个问题：表示层到底要解决什么需求，我们为何这样设计，用户到底可以说哪些命令，命令如何流入 pipeline，哪些工程细节值得强调，以及当前边界在哪里。

7.1 设计目标与表示层定位

若把整个系统的依赖方向写成

$$\text{CLI} \longrightarrow \text{Pipeline} \longrightarrow \{\text{I/O}, \text{Kernel}\} \longrightarrow \text{OS / CUDA Runtime},$$

那么 CLI 层只拥有第一条箭头。它知道用户给了哪些路径、哪些数值参数、是否要覆盖已有文件、是否要输出 JSON；但它不应该知道 `*.mat` 的内存映射细节，不应该读取矩阵元素，也不应该运行布局转置基准。表示层真正拥有的对象只有“用户意图”，因此它最合理的定位就是一个意图编译器（intent compiler）：

$$\text{user intent} \longrightarrow \texttt{CliOptions} \longrightarrow \texttt{PipelineConfig}.$$

从当前代码可反推，CLI 至少要满足四类互不重叠的需求。第一，它必须提供足够完整的表达能力，让用户能同时描述数据边界、数值参数、布局策略、运行控制与报告方式。第二，

它必须把高频路径压缩成短命令，例如只给输入文件时自动推导输出名，否则实验脚本会充满重复样板。第三，它必须在进入昂贵的 I/O 与 GPU 计算之前尽可能早地拒绝错误，把字符串错误固定在表示层，而不是把异常拖到 pipeline 或 kernel。第四，它必须是稳定契约，因为一旦文档示例、批处理脚本与论文实验开始依赖这些参数， CLI 就成为事实上的用户空间接口。

这四类需求共同解释了当前实现为何采用“表驱动解析 + 归一化 + 验证 + 显式映射”的结构。单纯把 `argc/argv` 直接塞给下层当然也能“跑起来”，但那样会让表示层失去边界：路径推断散落在执行代码里，错误信息靠异常偶遇，默认值无法审计，实验参数也难以复现。对于一个 CUDA 数值系统，这种混乱不是入口小问题，而是整个工程语义会漂移的起点。

7.2 命令空间与选项契约

当前 `ArgParser` 不是一串散落的 `if/else`，而是由 `OptionDefinition` 定义表驱动。长选项查找、短选项查找、是否需要值、帮助文本生成，全部共享同一张表。因此 CLI 的命令面并不是文档外加代码的双份描述，而是由数据结构直接定义的单一事实来源（single source of truth）。这点很关键，因为一旦接口扩展，最怕的不是选项变多，而是帮助文本、解析逻辑、默认语义三处开始不同步。

表 1: 当前 CLI 命令面与选项契约

类别	选项或形式	参数	语义
命令形式	`<input> [output]` `--`	路径或无	`<input>` 必填；`[output]` 省略时默认写入输入目录，文件名为 `<input-stem>.svd.<ext>`。显式 `--input/--output` 优先；`--` 之后 token 全按位置参数处理；多余位置参数报错。
数据边界	`--input/-i` `--output/-o`	`PATH`	输入路径必须存在且为文件；输出路径不能是目录，且规范化后不得与输入路径相同。输出已存在时必须显式给出 `--force/-y`。

格式契约	`--input-format`
	`--output-format`
	`--format/-f`
		`FMT`	`FMT=auto|mat|txt`。`--format` 同时设置输入与输出格式。输入格式为 `auto` 且扩展名不可推断时直接报错；输出格式为 `auto` 时按“显式输入格式 → 输入扩展名 → `mat`”回退，并在缺失扩展名时自动补齐标准后缀。

数值配置	`--epsilon/-e`
	`--max-sweeps/-s`
	`--threads-per-block/-t`
		`NUM/N`	三项都要求正数（其中 `max-sweeps` 与 `threads-per-block` 为正整数）。默认值分别为 `1e-9`、`128`、`256`。

续下页

表 1（续）当前 CLI 命令面与选项契约

类别	选项或形式	参数	语义
布局策略	`--layout-transpose-mode|min-cols|min-elems`	`MODE/N`	选项族表达（family notation）：按“前缀 + 后缀”展开为完整选项。`MODE=auto|on|off`。两个阈值都要求正数，主要用于 `auto` 模式决策；默认值为 `auto`、`16`、`4096`。
自动调优	`--layout-transpose-auto-tune|bench-reps|bench-sweeps`	无或 `N`	选项族表达（family notation）：按“前缀 + 后缀”展开为完整选项。开启后在 pipeline 前运行微基准自动调阈值；`bench-reps/bench-sweeps` 要求正整数，默认 `2/8`。仅当 `mode=auto` 时该调优实际生效，并用推荐值覆盖本次运行阈值。

运行控制	`--queue-capacity/-c`
	`--force/-y`
	`--dry-run`
	`--print-config`
		`N` 或无	`queue-capacity` 要求正整数，默认 `4`；`--force` 允许覆盖已有输出。 `--dry-run` 只做参数校验并打印有效配置后退出（不执行 pipeline）；`--print-config` 会先打印配置再继续执行。

报告与动作	`--json-report`
	`--quiet/-q`
	`--help/-h`
	`--version/-v`
		无	`--quiet` 仅抑制文本报告；`--json-report` 额外输出 JSON，二者可组合得到纯机器可读输出。 `--help/--version` 属于动作命令：直接输出并以退出码 `0` 返回，不进入运行配置校验与 pipeline 执行。

表 1 给出了当前 CLI 的完整命令面。这里有三点需要特别指出。第一，位置参数 `<input> [output]` 只是显式 `--input` 与 `--output` 的简写形式，因此

$$\texttt{jacobi-svd-cuda a.mat b.mat} \equiv \texttt{jacobi-svd-cuda --input a.mat --output b.mat}.$$

第二，`--help` 与 `--version` 是动作，不是运行配置；它们请求的是“输出帮助/版本后退出”，而不是构造一个特殊 pipeline。第三，布局转置相关参数被完整暴露出来，说明 CLI 不只是功能开关面板，也承担实验参数记录的职责。对 CUDA 程序而言， `--layout-transpose-mode`、阈值和自动调优开关都属于可复现状态，而不是可有可无的 UI 细节。

7.3 执行链路：从 argv 到 run_pipeline

当前控制流可以压缩为一条清晰的转换链：

$$\begin{aligned} argv &\xrightarrow{\text{token scan}} \texttt{ParseResult} \xrightarrow{\text{normalize}} \texttt{CliOptions} \\ &\xrightarrow{\text{validate}} \texttt{CliOptions} \xrightarrow{\Phi} \texttt{PipelineConfig} \xrightarrow{\text{run}} \texttt{PipelineReport}. \end{aligned}$$

若展开为顺序流程，当前 CLI 大致经历六步。

1. **词法扫描。** `ArgParser::parse` 对 `argv` 做一次线性扫描，把 token 分成长选项、短选项、位置参数与 `--` 终止符。长选项支持 `--name=value` 与 `--name value`；短选项支持 `-qv` 这样的无值聚合，也支持 `-e1e-10` 或 `-e 1e-10` 这样的带值形式。

2. **语义归约。** 无论 token 来自长选项还是短选项，最终都经由 `apply_option` 归约到同一份 `ParseResult`，从而保证 `-i` 与 `--input` 只对应一种内部语义。

3. **动作分流。** 解析结果先落入

$$\texttt{ParseResult} = (\texttt{ParseAction}, \texttt{CliOptions}),$$

   其中 $\texttt{ParseAction} \in \{\texttt{run}, \texttt{help}, \texttt{version}\}$。只有 `run` 才进入归一化与验证；帮助和版本动作直接输出并成功返回。

4. **归一化。** `normalize_run_options` 负责把不完整命令补成稳定配置：若输入格式为 `auto`，则尝试从扩展名推断；若输出路径为空，则生成

```
<input-parent>/<input-stem>.svd.<ext>;
```

   若输出格式仍未显式给出，则用输入格式或扩展名推导 fallback，并在必要时补上规范扩展名。

5. **运行前验证。** `validate_run_options` 检查输入是否存在、输入和输出是否为目录、输入输出是否规范化后相同，以及输出文件已存在时是否显式给出 `--force`。这一步的目标是把所有“本不该进入 pipeline 的错误”挡在入口。

6. **执行与报告。** `main` 先截获 `dry-run` 和 `print-config`，随后把 `CliOptions` 映射为 `PipelineConfig`，调用 `run_pipeline`，最后按 `quiet` 与 `json-report` 的组合输出文本摘要或 JSON 报告。

从复杂度看，这条流程非常便宜。若 token 数为 N、选项定义数为 M，则解析阶段的查找代价近似为

$$O(NM).$$

由于当前 M 很小，实际表现接近线性。入口层的成本因此几乎可以忽略，真正昂贵的仍然是文件 I/O、CUDA kernel 和结果写回。

7.4 关键工程细节

第一，表驱动解析保证接口集中。 `OptionDefinition` 同时承载内部标识、长名、短名、是否需要参数、值占位符与帮助描述。这样做的价值不是少写几行代码，而是把“CLI 到底向用户承诺了什么”收敛到一份可审计数据。对后续维护而言，这远好于把命令含义散落在解析逻辑、帮助文本和错误消息里。

第二，字符串不会泄漏到应用层。 当前实现把类型转换压缩进若干小函数，把自由字符串收缩为封闭集合（closed set）、拒绝部分解析、非正数和非有限值。这样一来，下游看到的不是“也许合法的字符串”，而是已经满足前置条件（precondition）的强类型参数。

第三，默认值服务高频路径，但不篡改显式意图。 `CliOptions` 的默认值把最常见运行压缩成一条短命令：

$$\epsilon = 10^{-9}, \qquad \texttt{max_sweeps} = 128,$$

$$\texttt{threads_per_block} = 256, \quad \texttt{queue_capacity} = 4,$$

$$\texttt{layout_transpose_mode} = \texttt{auto},$$

$$\texttt{layout_transpose_min_columns} = 16,$$

$$\texttt{layout_transpose_min_elements} = 4096.$$

自动调优默认关闭，`bench-reps` 默认为 2，`bench-sweeps` 默认为 8。若用户只给输入文件，CLI 自动生成 `.svd` 输出名；但若用户已经显式给出输出路径，表示层不会粗暴重命名，只会在格式仍未明确时做最小必要补全。这类细节体现的是工程边界感：系统应该替用户省去机械重复，而不是替用户改写明确意图。

第四，表示层控制位不会泄漏到 pipeline。 `make_pipeline_config` 只把输入、输出、格式、队列窗口与 kernel 配置映射到 `PipelineConfig`。像 `dry_run`、`print_config`、`json_report`、`quiet`、`force_overwrite` 这类纯表示层控制位，都在 `main.cpp` 中被消费。这意味着下层不需要知道“用户是不是正在 dry-run”，只需要处理一份已经准备好的执行配置。这样的边界使应用层保持纯净，也让测试更容易定位。

第五，错误分类和报告通道是显式契约。 当前退出码被分成三类：

$$0 : \text{成功，或 help/version/dry-run 等非执行动作成功},$$

$$2 : \text{CLI 参数错误}, \qquad 1 : \text{执行失败}.$$

这使脚本可以区分“命令写错了”和“命令合法但运行失败了”。成功路径上，文本摘要与 JSON 报告也是两条解耦通道：

$$\texttt{--quiet --json-report} \Rightarrow \text{只输出机器可读报告}.$$

对实验自动化而言，这种清晰语义远比“反正 stdout 会打印点什么”更可靠。

7.5 当前实现的边界与可演化方向

当前 CLI 已经具备完整可用性，但边界也非常明确。第一，输入输出同一性判断仍基于 `absolute + lexically_normal` 的词法规范化（path normalization）。这能防住 `a.mat` 与 `./a.mat` 之类的常见误用，却不能完整识别硬链接、符号链接或平台路径别名。若未来进入更严苛环境，可以升级为平台文件身份比较，例如 POSIX 的 `st_dev/st_ino` 或 Windows file index。

第二，JSON 报告当前仍由手写 `std::cout` 直接拼接。由于现有字段几乎全是数值与固定枚举，风险尚可接受；但如果未来把路径、设备名称、错误消息或驱动字符串纳入 JSON，就必须补上最小字符串转义、稳定 schema 与版本字段。机器接口一旦被脚本依赖，就不应建立在“这个字段大概不会含引号”的侥幸上。

第三，当前命令来源只有 CLI 本身。这对可复现性是好事，因为一条命令就足以完整记录一次实验状态；但若未来加入配置文件，优先级必须清晰写成

$$\text{default} \prec \text{config file} \prec \text{CLI override},$$

并保留 `dry-run` 展示最终合并结果。否则，配置来源一多，实验者最容易失去对“这次到底按什么参数执行”的把握。

第四，`definitions_` 这张选项表还没有被进一步复用。实际上，它天然适合生成 shell 补全（shell completion）、接口文档，甚至参数兼容性检查。若后续命令面继续增长，这张表就是最自然的扩展支点。同理，当前解析阶段对长名和短名采用线性查找是完全可接受的；只有当命令数继续增长，或者未来真的引入子命令（subcommand）体系时，才值得把它演化为索引结构或分层命令模型。

第五，当前报告仍然是摘要型报告，而不是完整遥测（telemetry）。它告诉我们 testcase 数、总 sweep 数、布局转置运行模式与总耗时，却没有每阶段耗时、队列阻塞时间、重排序缓冲区峰值或 GPU kernel 明细。若未来要做严肃 profiling，更合理的方向是增设独立的遥测结构，而不是不断给稳定用户报告追加调试字段。

7.6 本节小结

这一章的核心是说明 `main.cpp` 如何建立一个边界清楚的表示层。用户写下的 `argv` 先被解析为 `ParseResult`，再经由归一化与验证收缩为 `CliOptions`，随后映射为 `PipelineConfig`，最终得到 `PipelineReport` 与退出码。整条路径的价值在于：字符串错误被尽早拒绝，默认值被集中管理，覆盖风险被显式化，实验参数可以被完整记录，而 pipeline 不必背负任何本属于表示层的偶然复杂度。

因此，CLI 在这个项目中不是外围装饰，而是系统契约的第一道强类型边界。它决定用户意图是否能被正确表达，决定错误是在入口被发现还是拖到 GPU 上爆炸，也决定实验脚本、文档示例与自动化流程能否长期稳定运行。对于一个 CUDA 数值程序，这类安静的入口代码和高吞吐 kernel 一样，都属于工程质量本身。

8 CMake 构建体系：把数值程序编译成可复现实验

前几章沿着算法、GPU kernel、I/O、pipeline 与 CLI 的方向逐层展开。本章讨论最后一层安静但关键的基础设施： `CMakeLists.txt`。它看起来只是几十行构建脚本，却实际承担着一个数值系统的实验契约（experimental contract）：哪些源文件被纳入可执行程序，哪些编译器前端被允许，哪些语言标准必须成立，二进制产物被放到哪里，以及 Windows、Linux 与 CUDA 工具链之间的差异如何被压缩到一个可理解的边界内。

在高性能计算（High Performance Computing, HPC）项目中，构建系统不是附属物。它决定研究者能否在另一台机器上复现实验，决定错误是在配置阶段暴露还是在运行时爆炸，也决定项目在从课程实现演进为论文 artifact 时是否还能保持可审计性（auditability）。若 `kernels.cu` 是数值内核，`CMakeLists.txt` 就是把数值内核、主机代码和平台约束缝合起来的编译协议。这个协议越小、越显式、越少魔法，后续的性能调优和工程扩展就越可靠。

8.1 构建系统的研究对象：从文件集合到可执行 artifact

本项目当前构建图（build graph）的输出只有一个可执行文件：

$$\mathcal{B} : \{\texttt{main.cpp}, \texttt{pipeline.cpp}, \texttt{io.cu}, \texttt{kernels.cu}\} \longrightarrow \texttt{jacobi-svd-cuda}.$$

其中 `include/` 提供接口边界，`src/` 提供实现边界，`build/` 承载最终二进制。构建系统的职责不是理解 SVD，也不是管理实验输入输出，而是保证这条映射在工具链允许的范围内尽可能确定。

从 MECE 的角度看，`CMakeLists.txt` 的职责可以分成四类，彼此不重叠且覆盖当前需求：

- **发现工具链**：找到 `nvcc`，或启用 CMake 原生 CUDA 语言（CUDA language）；
- **定义编译契约**：固定 C++20/CUDA C++20、包含目录、源文件集合与可分离编译（separable compilation）；
- **归一化平台差异**：处理 Windows 可执行扩展名、Visual Studio 多配置生成器（multi-config generator）输出路径，以及源文件编码；
- **暴露可复现入口**：通过同一个目标（target）`jacobi-svd-cuda` 支持 `cmake --build`，让实验脚本不依赖 IDE 的隐藏状态。

这四件事之外的内容，当前构建系统有意不做：不下载依赖，不猜测 GPU 架构，不自动运行 benchmark，不把论文构建和程序构建混在一起。这样的克制符合本项目的工程阶段。一个 CMake 文件越早变成“万能仪表盘”，越容易把平台问题、实验问题和领域问题搅在同一锅里，最后谁都说不清是哪一层坏了。

8.2 工具链选择与构建契约

最小语言声明：为什么从 `LANGUAGES NONE` 开始。 项目声明为：

```
project(jacobi_svd_cuda LANGUAGES NONE)
```

这不是无意的极简，而是一种延迟承诺（lazy commitment）。普通 CMake 项目常在 `project()` 中直接声明 `CXX CUDA`，让 CMake 立刻探测 C++ 与 CUDA 编译器。对跨平台 CUDA 项目而言，这个动作并不总是廉价。特别是在 Windows 上，CMake 原生 CUDA 语言模式往往依赖 Visual Studio CUDA toolset 与生成器状态；若开发者只想用 `nvcc` driver mode 快速构建，过早启用 CUDA 语言反而会把配置阶段绑定到更重的 IDE/toolset 组合。

因此，当前脚本先把项目置于无语言状态，再由选项决定构建路线：

$$\texttt{JACOBI_USE_CMAKE_CUDA_LANGUAGE} \in \{\texttt{OFF}, \texttt{ON}\}.$$

当该选项为 `OFF` 时，CMake 只负责发现 `nvcc` 并注册自定义构建命令；当该选项为 `ON` 时，才显式调用 `enable_language(CXX)` 与 `enable_language(CUDA)`。这把“是否进入 CMake CUDA 语言模型”从隐式环境假设变成了显式用户选择。

用形式化语言说，配置阶段被拆成两段：

$$\text{configure} = \text{base} \circ \text{select_toolchain}\,.$$

base 不要求 CUDA 语言可用，只解析路径、选项和平台；select_toolchain 根据用户选择进入更具体的编译模型。这个设计牺牲了一点 CMake target 纯度，换来更强的启动鲁棒性。对一个正在演化的 CUDA 数值项目，这种权衡是合理的：先保证可构建，再逐步收紧构建图。

两条编译路径：原生 CUDA 语言与 nvcc driver mode。 当前构建系统提供两条互斥路径。它们不是“高级”和“低级”的关系，而是面向不同工具链现实的两个前端。

第一条路径：CMake 原生 CUDA 语言。 当 JACOBI_USE_CMAKE_CUDA_LANGUAGE=ON 时，CMake 把 .cu 视为一等源文件：

```
enable_language(CXX)
enable_language(CUDA)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CUDA_STANDARD 20)

add_executable(jacobi-svd-cuda ${JACOBI_SOURCES})
    target_include_directories(jacobi-svd-cuda PRIVATE "${CMAKE_SOURCE_DIR}/include")
set_target_properties(jacobi-svd-cuda PROPERTIES CUDA_SEPARABLE_COMPILATION ON)
```

这条路线的优势是 target 语义更完整。CMake 知道哪些文件属于 CUDA 编译单元，知道如何把 CUDA 编译、设备链接（device linking）和宿主链接（host linking）组织进构建图，也能更自然地接入 target_compile_features、target_compile_options、安装导出（install/export）与测试框架。若项目未来拆分为库目标和多个工具，原生 CUDA 语言应当成为主路径。

这里启用 CUDA_SEPARABLE_COMPILATION 是防御性选择。可分离编译允许设备代码跨翻译单元（translation unit）组织，并触发必要的设备链接阶段。当前 kernels.cu 尚未形成复杂的多 CUDA 翻译单元结构，但提前把目标属性放在可见处，能避免未来把设备函数拆出文件时遇到隐式链接失败。

第二条路径：nvcc driver mode。 默认路径不启用 CMake CUDA 语言，而是直接寻找 nvcc：

```
find_program(JACOBI_NVCC_EXECUTABLE
NAMES nvcc
HINTS "$ENV{CUDA_PATH}/bin"
REQUIRED)
```

随后 CMake 注册一个自定义命令，把所有源文件交给 nvcc：

```
add_custom_command(
OUTPUT "${JACOBI_EXE_PATH}"
```

```
        COMMAND "${CMAKE_COMMAND}" -E make_directory "${JACOBI_OUTPUT_DIR}"
        COMMAND "${JACOBI_NVCC_EXECUTABLE}" ${JACOBI_NVCC_ARGS}
    ${JACOBI_SOURCES} -o "${JACOBI_EXE_PATH}"
        DEPENDS ${JACOBI_SOURCES}
    VERBATIM
    )

    add_custom_target(jacobi-svd-cuda ALL DEPENDS "${JACOBI_EXE_PATH}")
```

nvcc driver mode 的核心思想是：让 NVIDIA 编译器驱动同时协调 CUDA 编译与宿主 C++ 编译。它绕开了某些 CMake/Visual Studio/CUDA toolset 组合上的配置摩擦，使项目在安装了 CUDA Toolkit 的环境中更容易先跑起来。这条路线也非常适合小型单可执行程序：源文件集合明确，外部依赖为零，链接关系不复杂。

但它的代价也必须诚实写出来。自定义命令不是完整的 CMake target 模型；CMake 对单个源文件的语言属性、逐目标编译选项、IDE 源文件分组、依赖扫描和安装导出都掌握得更少。换言之，默认路径是一条现实主义路径，不是最终的构建架构。它服务于当前阶段的首要目标：让 C++20/CUDA 12.x 代码在 Windows 与类 Unix 环境中以最低摩擦产出同名可执行文件。

源文件集合：显式列表胜过目录通配。 当前脚本显式列出源文件：

$$\mathcal{S} = \{\texttt{src/main.cpp}, \texttt{src/pipeline.cpp}, \texttt{src/io.cu}, \texttt{src/kernels.cu}\}.$$

这比 `file(GLOB ...)` 更朴素，也更适合研究代码。显式列表具有三个优点：

- **审计性**：读者打开 `CMakeLists.txt` 就能知道 artifact 由哪些实现文件组成；
- **确定性**：新文件不会因为放进 `src/` 就悄悄进入构建，避免未完成实验代码污染结果；
- **变更可见性**：新增或删除编译单元会反映为构建脚本 diff，代码评审时不会被隐藏。

这正是“好品味”的一个小例子：不要让特殊情况藏在工具的自动行为里。对于 Linux kernel 式的工程直觉而言，构建图应该像数据结构一样可读。若一个文件参与二进制，它就应当在构建描述中占有明确位置；若它只是实验草稿，它就不应被通配规则自动拖进链接器。

当然，显式列表也有维护成本：新增源文件时必须更新 CMake。这个成本在当前规模下是正收益，因为它强迫开发者确认模块边界。只有当项目增长到数十个子目录、多个库目标和自动生成文件时，才需要重新评估是否引入更结构化的 `add_subdirectory()` 层次。

输出路径契约：把 artifact 放在项目根 `build/`。 当前输出目录固定为：

$$\texttt{JACOBI_OUTPUT_DIR} = \texttt{\${CMAKE_SOURCE_DIR}/build}.$$

Windows 下输出 `jacobi-svd-cuda.exe`，其他平台输出 `jacobi-svd-cuda`。这形成了一个稳定用户契约：

$$\text{build artifact} = \begin{cases} \texttt{build/jacobi-svd-cuda.exe}, & \text{Windows}, \\ \texttt{build/jacobi-svd-cuda}, & \text{otherwise}. \end{cases}$$

从纯 CMake 教义看，把运行产物放到源码根下的 `build/` 而不是 `${CMAKE_BINARY_DIR}` 并不完美。它让“CMake 的二进制目录”和“项目的可执行输出目录”分离，也可能在多构建目录并行时产生覆盖。可是从本项目的实验工作流看，这个决定有现实价值：CLI 文档、实验脚本和人工运行都可以稳定引用同一位置，而不必知道 CMake 生成目录叫 `cmake-build-debug`、`out/build/x64-release` 还是别的 IDE 私有名字。

原生 CUDA 语言路径还额外处理了多配置生成器：

```
foreach(config Debug Release RelWithDebInfo MinSizeRel)
string(TOUPPER "${config}" config_upper)
    set(CMAKE_RUNTIME_OUTPUT_DIRECTORY_${config_upper} "${JACOBI_OUTPUT_DIR}")
endforeach()
```

这是 Windows/Visual Studio 生态中特别重要的一步。单配置生成器（single-config generator）通常在配置时选择 `CMAKE_BUILD_TYPE`；多配置生成器则在构建时选择 `Debug` 或 `Release`，并倾向于把产物放进配置子目录。当前脚本把这些配置统一投影到同一 `build/` 目录，牺牲了同时保存多种配置产物的能力，换取命令行层的稳定路径。

这是一种有意识的收敛：本项目不是库发行包，而是单工具数值程序。它更关心“用户下一条命令能否稳定找到程序”，而不是“一个构建树里是否长期保留四种优化级别的二进制”。若未来进入 CI 矩阵测试或性能对照阶段，可以把输出路径改成

$$\texttt{build/\${config}/jacobi-svd-cuda}$$

并同步更新实验脚本；在那之前，单一 artifact 路径更符合项目现实。

语言标准与编码：让编译器拒绝隐式漂移。 两条路径都要求 C++20。原生 CUDA 语言路径显式设置：

$$\texttt{CMAKE_CXX_STANDARD} = 20, \qquad \texttt{CMAKE_CUDA_STANDARD} = 20,$$

并关闭扩展：

$$\texttt{CMAKE_CXX_EXTENSIONS} = \texttt{OFF}, \qquad \texttt{CMAKE_CUDA_EXTENSIONS} = \texttt{OFF}.$$

默认 `nvcc` driver mode 则传入：

$$\texttt{-std=c++20}.$$

这不是语法偏好，而是接口契约。项目 I/O 层依赖现代 C++ 标准库能力，CUDA 主机代码也与 C++20 编译模型绑定。若某台机器悄悄用较旧标准编译，失败应当发生在编译期，而不是以更隐蔽的模板错误、ABI 差异或运行时行为漂移出现。

Windows 上还额外传入：

$$\texttt{-Xcompiler=/utf-8}.$$

这是一个很小但非常关键的跨文化工程细节。Windows PowerShell 与 MSVC 历史上经常受到本地代码页影响；在中文环境中，默认编码可能是 GBK。项目源文件与 LaTeX 文档都应以 UTF-8 为最终状态。对宿主编译器显式指定 `/utf-8`，可以避免中文注释、字符串字面量或诊断信息在编译阶段被错误解释。它不改变算法，却保护了项目的长期可读性。对于 Klee 这样的中国大陆开发环境，这类细节不是装饰，是 reproducibility 的一部分。

包含目录与依赖边界：当前只有项目内头文件。 当前唯一包含目录为：

$$\texttt{include/}.$$

这意味着构建系统没有外部三方依赖解析，没有包管理器，没有下载步骤，也没有系统库探测。这样的空依赖面非常有价值：若构建失败，故障空间主要集中在 CMake、CUDA Toolkit、宿主编译器和项目源代码四类，而不是扩散到复杂依赖森林。

可以把当前依赖关系写成：

$$D = D_{\text{toolchain}} \cup D_{\text{stdlib}} \cup D_{\text{cuda}} \cup D_{\text{project}}.$$

其中 D_{project} 只包含 `include/` 与 `src/`。项目没有引入 BLAS、cuSOLVER 或第三方 CLI 库，这与前文的教学目标一致：我们要显式展示单边雅可比 SVD 如何从矩阵格式、pipeline 到 CUDA kernel 端到端运行，而不是把核心行为隐藏在大型库调用之后。

这并不意味着“永远不要依赖”。如果未来要做性能对照，引入 cuSOLVER 作为基线（baseline）是合理的；如果要做严肃测试，引入 GoogleTest 或 Catch2 也合理。但依赖应当服务于明确研究问题，而不是为了显得工程化。每一个新依赖都会增加供应链状态、版本约束和跨平台失败模式；它必须带来足够收益。

8.3 可复现实验的操作模型

构建命令：可复现实验的最小仪式。 在默认路径下，一个干净构建可以写为：

```
cmake -S . -B build-cmake
cmake --build build-cmake
```

产物会被写入项目根的 `build/`。若要尝试 CMake 原生 CUDA 语言路径：

```
cmake -S . -B build-cmake-cuda -DJACOBI_USE_CMAKE_CUDA_LANGUAGE=ON
cmake --build build-cmake-cuda --config Release
```

这两组命令揭示了构建系统对用户的最小承诺：配置目录可以变化，生成器可以变化，但用户目标名与可执行输出位置保持稳定。在论文 artifact 或课程复现实验中，这种稳定性会直接降低沟通成本。读者不必理解项目内部每个 CMake 变量，只需要知道：先 configure，再 build，然后运行 `build/jacobi-svd-cuda`。

更进一步，构建命令本身应当出现在实验记录中。对数值程序而言，仅记录输入矩阵和运行参数是不够的；编译路径同样影响结果。例如 `Debug` 与 `Release` 的性能不可比较，`nvcc` 版本差异可能影响优化策略，不同 GPU 架构也可能改变浮点执行细节。严肃报告至少应记录：

$$(\text{CMake version}, \text{CUDA Toolkit version}, \text{host compiler}, \text{build type}, \text{GPU model}).$$

当前 CMake 文件尚未自动生成这些元数据，但 CLI 的 JSON 报告未来可以接收它们。构建系统和运行系统在这里会合：前者定义 artifact，后者报告 artifact 的行为。

正确性不变量：构建系统应保证什么。 可以为当前构建系统提炼出五个不变量（invariant）。这些不变量比具体 CMake 语法更重要，因为它们描述了项目希望长期保持的工程性质。

不变量一：源文件集合完整且最小。 每一次构建必须包含 CLI、pipeline、I/O 与 CUDA kernel 四个实现单元；同时不应包含未声明的实验源文件。形式化地说：

$$\text{sources}(\texttt{jacobi-svd-cuda}) = \mathcal{S}.$$

若未来新增测试、benchmark 或工具程序，应通过新 target 表达，而不是悄悄塞入主程序源集合。

不变量二：语言标准不随环境漂移。 所有有效构建都应在 C++20/CUDA C++20 语义下进行。若工具链不支持，应配置或编译失败。静默降级到 C++17 是错误状态，不是兼容模式。

不变量三：artifact 路径对用户稳定。 不论 CMake 生成目录如何命名，用户可执行文件都位于 `build/jacobi-svd-cuda(.exe)`。这支撑文档示例、实验脚本和 CLI 章节中的运行契约。

不变量四：Windows 编码显式为 UTF-8。 只要通过 `nvcc` driver mode 在 Windows 编译，宿主编译器就应接收 `/utf-8`。任何依赖本地代码页的构建都不可复现，也不尊重中文源文件的长期维护。

不变量五：工具链选择是显式的。 用户通过 `JACOBI_USE_CMAKE_CUDA_LANGUAGE` 选择构建前端。构建系统不应基于偶然环境状态偷偷在两条路径之间切换，因为这会让同一命令在不同机器上产生难以解释的行为。

失败模型：把错误尽早放到正确层。 好的构建系统不只是让成功路径舒服，也要让失败路径可解释。当前设计的失败模型可以分为四层：

失败层次	典型原因	理想暴露位置
工具发现	找不到 `nvcc` 或 CUDA Toolkit	CMake 配置阶段
语言能力	编译器不支持 C++20/CUDA C++20	配置或编译早期
源码错误	头文件缺失、模板错误、CUDA API 使用错误	编译阶段
链接错误	设备链接、宿主链接或运行库解析失败	链接阶段

这个分层很重要。若没有 `find_program(... REQUIRED)`，用户可能在构建中途才看到模糊的命令不存在错误；若不固定 C++20，用户可能被几十行模板展开淹没，而真正原因只是语言标准不对；若输出路径没有统一，运行脚本可能报告“找不到程序”，但真正失败点在构建目录约定不一致。

工程上要追求的不是“永不失败”，而是“在最接近根因的位置失败”。这和前文 CLI 的设计哲学一致：参数错误应在 CLI 层拒绝，矩阵格式错误应在 I/O 层拒绝，CUDA runtime 错误应在 kernel 边界转换为异常；构建错误也应在配置或编译阶段以局部原因暴露。

8.4 工程审查与后续演化

复杂度审查：当前 CMake 的好味道与坏味道。 从 Linux kernel 式的审查口味看，当前构建脚本有几处好味道。

第一，控制流浅。顶层只有一个二分选择：原生 CUDA 语言或 `nvcc` driver mode。没有嵌套平台矩阵，没有复杂函数宏，也没有把 CMake 写成另一门大型程序。这使读者可以在一分钟内建立完整心理模型。

第二，路径变量集中。输出目录、源文件集合、可执行路径和 `nvcc` 参数都以 `JACOBI_` 前缀变量表达。这降低了字符串重复，也让未来迁移到多 target 时有明确切入点。

第三，默认路径务实。对 Windows CUDA 初学者而言，CMake 原生 CUDA 语言经常把问题引向 Visual Studio 组件安装、生成器选择和 CUDA integration 细节。默认使用 `nvcc` driver mode，能让更多用户先得到可执行程序。

但也存在需要诚实面对的坏味道。

第一，自定义命令降低了 target 语义。 `add_custom_target` 产出的 `jacobi-svd-cuda` 不是普通 `add_executable` target。这意味着很多 CMake 生态能力不能自然附着其上。若未来需要安装、导出、测试依赖、逐源文件属性或 IDE 集成，默认路径会开始拖后腿。

第二，依赖集合没有包含头文件。 `add_custom_command(DEPENDS ...)` 当前依赖源文件，但没有显式列出 `include/*.hpp`。如果只修改头文件，某些生成器可能不会重新触发自定义命令。这是实际风险。短期可以把头文件加入 `JACOBI_SOURCES` 旁的 `JACOBI_HEADERS` 并加入 `DEPENDS`；长期则应迁移到原生 target，让编译器依赖扫描接管。

第三，GPU 架构没有显式记录。 当前没有设置 `CMAKE_CUDA_ARCHITECTURES` 或 `-arch=sm_XX`。这保持了简单，但会让不同机器上的 `nvcc` 默认行为影响生成代码。若要进入严肃 benchmark，应显式选择架构，例如面向特定实验 GPU 固定 `sm_89` 或同时生成若干架构的 fat binary（fat binary）。

第四，构建类型没有进入默认 `nvcc` 路径。 原生 CMake target 能自然区分 `Debug`、`Release` 与 `RelWithDebInfo`；自定义 `nvcc` 命令当前没有根据构建类型添加 `-G`、`-O3`、`-lineinfo` 等选项。这对功能正确性影响不大，但会影响性能解释。论文式实验必须明确编译优化级别，否则吞吐数据缺少上下文。

第五，源码根 `build/` 作为输出目录可能与 CMake 构建目录混淆。 如果用户执行 `cmake -S . -B build`，CMake 自己的构建树和项目可执行输出目录会重合。当前项目已经有根 `build/`，所以这在实践中可能发生。文档应鼓励使用 `build-cmake` 或 `out/build` 作为 CMake 二进制目录，把根 `build/` 留给最终 artifact。

这些问题都不是灾难，但它们标出了演化方向。真正危险的不是存在坏味道，而是对坏味道没有边界感。现在我们知道它们在哪里，就能在项目规模增长时逐一消除。

可复现性模型：构建是实验状态的一部分。 数值实验的可复现性（reproducibility）通常被理解为“相同输入得到相同输出”。但对于 CUDA 程序，更完整的状态应写成：

$$y = f(x; \theta_{\text{alg}}, \theta_{\text{cli}}, \theta_{\text{build}}, \theta_{\text{device}}).$$

其中 x 是输入矩阵，θ_{alg} 是算法参数，例如 ϵ 与最大 sweep 数；θ_{cli} 是 CLI 层选项；θ_{build} 包含编译器、标准、优化级别和 GPU 架构；θ_{device} 包含实际 GPU、driver 与 runtime 状态。

构建系统至少控制 θ_{build} 的一部分。当前脚本已经固定：

$$\theta_{\text{build}} \supset \{\texttt{C++20}, \texttt{CUDA C++20}, \texttt{source set}, \texttt{include dir}\}.$$

尚未固定的部分包括：

$$\{\texttt{build type}, \texttt{optimization flags}, \texttt{CUDA architecture}, \texttt{compiler version metadata}\}.$$

如果本文档要接近顶会 artifact 的质感，不能只展示“如何构建”，还要说明“哪些自由度尚未被固定”。这不是自我削弱，而是科学诚实。一个系统在可控变量和未控变量之间划线越清楚，别人复现时越知道应该检查哪里。

与论文构建的关系：两个构建系统，不要互相污染。 仓库中还有 `tex/latexmkrc`，用于构建本文档。它和根目录 `CMakeLists.txt` 服务于不同 artifact：

$$\texttt{CMakeLists.txt} \longrightarrow \texttt{jacobi-svd-cuda},$$

$$\texttt{tex/latexmkrc} \longrightarrow \texttt{main.pdf}.$$

这两个构建系统应该保持分离。程序构建需要 CUDA Toolkit、宿主编译器和平台链接器；文档构建需要 XeLaTeX、中文字体和 LaTeX 包。把它们强行合并进一个 CMake 超级构建会制造不必要耦合：用户只想编译程序时不应被 TeX 环境阻塞，用户只想读 PDF 时也不应安装 CUDA。

更好的做法是让二者在发布层会合，而不是在构建层纠缠。例如未来可以用 CI 定义两个 job：

$$\texttt{build-program}, \qquad \texttt{build-paper}.$$

二者共享同一 commit，但依赖环境不同，失败也互不污染。若需要生成完整 release artifact，再由上层工作流收集 `jacobi-svd-cuda` 与 `main.pdf`。这种分离体现了软件工程里的单一职责原则（single responsibility principle）：每个构建系统只对自己的 artifact 负责。

面向未来的 target 化重构。 当前 CMake 文件的下一步演化，不应是堆更多变量，而应是把构建图从“单命令”提升为“target 化架构”。一个合理方向是：

$$\begin{aligned}
\texttt{jacobi-svd-core} &: \text{I/O} + \text{pipeline} + \text{kernels},\\
\texttt{jacobi-svd-cli} &: \text{main.cpp} + \text{core},\\
\texttt{jacobi-svd-tests} &: \text{unit tests} + \text{core},\\
\texttt{jacobi-svd-bench} &: \text{benchmarks} + \text{core}.
\end{aligned}$$

此时主程序不再直接拥有所有源文件，而是链接核心库：

$$\texttt{jacobi-svd-cli} \longleftarrow \texttt{jacobi-svd-core}.$$

这样做有三类收益。第一，测试可以绕过 CLI，直接调用 I/O、pipeline 或 kernel 包装接口；第二，benchmark 可以共享同一核心实现，避免复制源文件集合；第三，安装和导出可以围绕库 target 组织，而不是围绕自定义命令拼接。

但重构也要把握时机。当前项目只有一个可执行程序，过早拆出多 target 可能让读者在构建层消耗过多认知。更好的策略是：当测试目录出现、benchmark 需要稳定入口、或核心代码被第二个工具复用时，再进行 target 化。复杂度应由真实需求驱动，而不是由“看起来专业”的架构冲动驱动。

CI 集成：从可构建到可验证。 持续集成（Continuous Integration, CI）不是简单地“在云上跑一下 CMake”。对 CUDA 项目而言，CI 至少有三种层级：

- **配置级 CI**：验证 CMake 能发现工具链、生成构建图；
- **编译级 CI**：验证源代码能在指定 CUDA/host compiler 组合下生成 artifact；
- **运行级 CI**：在真实或虚拟 GPU 上运行小矩阵测试，检查 SVD 残差与正交性。

当前仓库还没有 CI 配置，因此本章只能给出设计边界。最小 CI 不应追求覆盖所有 GPU，而应先固定一个已知环境，例如 Windows + CUDA 12.8 或 Ubuntu + CUDA 12.x。运行级测试可以选择小矩阵：

$$A \in \mathbb{R}^{4\times 3}, \quad A \in \mathbb{R}^{8\times 8},$$

检查

$$\frac{\|A - U\Sigma V^T\|_F}{\|A\|_F} < \tau, \qquad \|V^T V - I\|_F < \tau.$$

这样 CI 才真正服务于数值系统，而不是只证明“编译器今天没有报错”。

CI 还应显式保存构建日志和版本元数据。对 CUDA 项目来说，`nvcc --version`、`cmake --version` 和 GPU 型号不是噪声，而是实验上下文。一个顶会 artifact 的读者并不只想知道结果是否通过；他还想知道结果在哪个环境下通过。

安装、导出与包边界：现在不做，但要知道为什么。 当前项目没有 `install()` 规则，也没有 CMake package export。这是合理的，因为本项目的主要交付物是一个端到端 CLI 工具和技术文档，而不是供其他项目链接的库。过早设计安装前缀、版本命名空间、导出 target 和 ABI 策略，会把注意力从算法正确性与实验复现转移到发行工程。

但如果未来要把核心 SVD 实现作为库发布，就必须重新设计边界：

- 公共头文件应区分 public/private include；
- 库 target 应提供稳定命名空间，例如 `jacobi::svd`；
- CUDA runtime 依赖与编译架构应进入导出配置；

- ABI 兼容策略必须明确，尤其是结构体布局、异常边界与 C++ 标准库类型暴露。

这里再次回到“不破坏用户空间”的原则。一旦安装导出被发布，外部用户就可能把它写进自己的构建系统。此后任何 target 名称、头文件路径或链接语义的变化都可能成为破坏性变更。当前不做 install/export，是为了避免过早承诺尚未稳定的库 ABI。

8.5 小结

`CMakeLists.txt` 在本项目中不是机械脚本，而是连接研究代码与可复现实验的协议层。它把四个实现文件、 C++20/CUDA C++20 语言标准、`include/` 接口目录、Windows 编码约束和稳定输出路径组织成一个可执行 artifact。它还通过 `JACOBI_USE_CMAKE_CUDA_LANGUAGE` 暴露两条工具链路径：一条面向 CMake 原生 CUDA target 生态，一条面向现实环境中更容易成功的 `nvcc` driver mode。

从系统论文的角度看，本章的核心结论并不是“CMake 应该怎么写”，而是“构建系统应当把哪些实验自由度显式化”。当前实现已经固定了源文件集合、语言标准、包含目录、输出位置和基本平台差异；它尚未完全固定优化级别、GPU 架构、头文件依赖扫描和版本元数据。这些边界清楚地告诉我们：哪些结果现在可以复现，哪些结果在进入严肃 benchmark 前还需要收紧。

最好的构建系统不是最大的构建系统，而是把复杂度放在正确层次上的构建系统。对于 `jacobi-svd-cuda`，当前 CMake 的价值正在于它足够小、足够显式、足够容易审查。它让读者把注意力放回真正重要的问题：单边雅可比 SVD 如何映射到 GPU，矩阵如何稳定进出系统，pipeline 如何隔离计算与写出，以及一个数值程序如何从代码变成可被别人重新运行、重新测量、重新质疑的科学 artifact。